New Designer Handbook

How we design Sauce

Internal Use Only

Links of Interest

Brand

Branding Elements

Brand Style Guide

Illustrations Catalog

Brand Storybook

Company

Sauce Labs Company Overview

Product Design Wiki

YouTube Channel

Sauce Bot - The Beginning

Twitter

Corporate Messaging

Overview of Sauce Labs

Github Training Repo

Cookbook

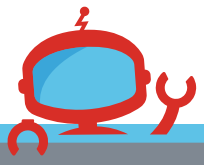
Engineering Team

IT

IT Home

To submit a help request, send an email to: helpdesk@saucelabs.com.

Helpdesk Schedule:



US (PST)		EU (CEST)		CONTACT
3:00 AM	8:00 AM	12:00 PM	5:00 PM	Chelsea
8:00 AM	1:00 PM	5:00 PM	10:00 PM	Navraj
1:00 PM	5:00 PM	10:00 PM	2:00 AM	Wathiq

HR

Do you have a question for HR? Submit your request to: hr@saucelabs.com.

Our Team

Brenda Stine, Director

a.k.a “bossman,” Brenda works out of the SF office. She enjoys dance, making cool things, and rap battles.

Seth Kornfeld, Senior UX Designer

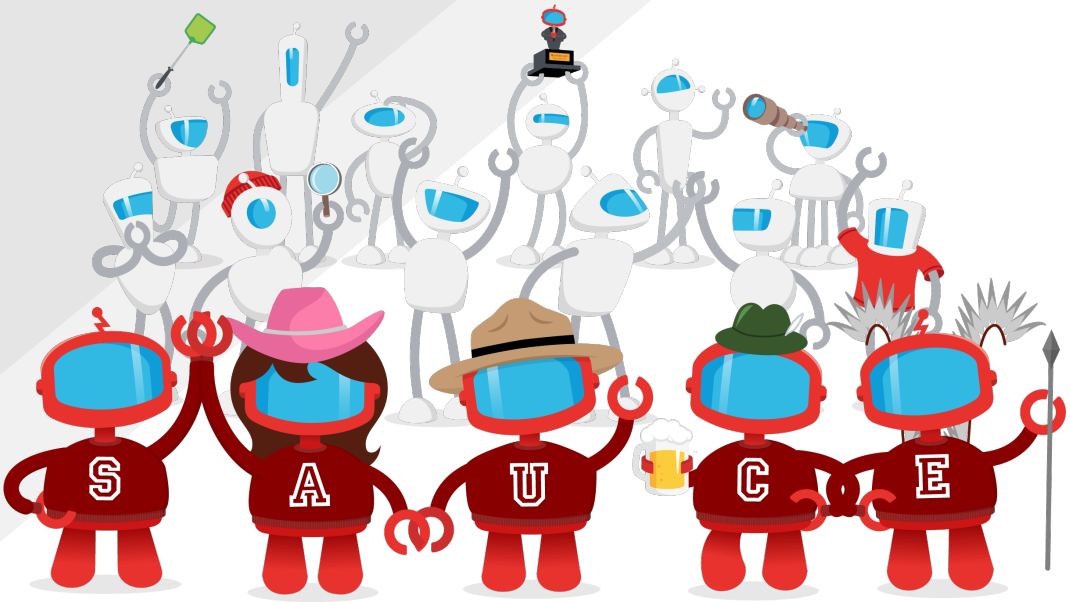
a.k.a “child of the korn,” Seth lives in Seattle and works out of the Vancouver office remotely. He enjoys cooking, gardening, reading, and showing up late to meetings.

Josh King, Principal UX Designer

a.k.a “the shorter King at Sauce,” Josh lives in Montana and works remotely and truly hates writing in the 3rd person.

Julie McGehee, Lead User Researcher

Passionate about technology, research, converting insights into knowledge, and learning and growing from diverse cultures. She’s known as Julie for now.



Tooling

Reporting and Analytics

Mixpanel

Event tracking, customer journey tracking

Looker

Informed data, more overall customer reporting

Google Analytics

Page tracking analysis

Mailchimp

Surveys, customer emails

Productivity

GSuite

Email, spreadsheets, docs, forms (for surveys) and Drive (may your Finder rest in peace)

Zoom

Online meetings

Aha!

Product team backlog and planning

Jira

Tickets

Figma

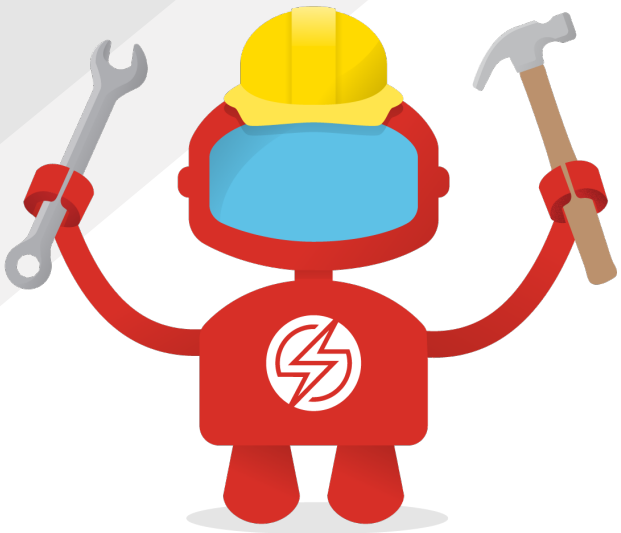
Designer's IDE

Miro

Collaboration

Adobe Creative Cloud

Adobe products & fonts



Important Note
Make sure you have all the tools you need. Some of these will be set up for you, but in case they are not, feel free to put in a helpdesk ticket to request it.

Design System

At the core of every modern software company is a design system. I'd like to introduce you to Roux. Roux is used to thicken sauces and is a building block many different types of sauce. The term "Roux" was coined by our own Alistair Heyes. Our system is comprised of three main parts: Our Asset Library, our Component Library, and our Style Guide.

Asset (Pattern) Libraries

Our pattern libraries are housed in Figma. To access the library, click on the Assets in the top left panel, then click on the book icon "Team Library." From this panel, add whichever items you need.

Atomic Design

We are moving towards an Atomic structure. Many pieces of this are in play, and we are slowly moving towards this philosophy.

Foundations and Principles

Accessibility, typography, colors, grid information.

View in wiki

Component Library

Our front-end React component library can be viewed via Storybook (requires VPN).

View Storybook

Style Guide

Tech Content Guidelines

Guidance on writing product copy and documentation.

View TCD Guidelines

Editorial Guidelines

Contains style, tone, grammar, SauceBot usage, and many other guidelines around our content.

View Sauce Labs Editorial Style Guiide

Using the Design System

Enabling Libraries in your File

Below, we go over how to create a new Figma component and add it to the pattern library.

1. Open the **Assets Panel.**
2. Click the 📖 **Team Library** icon.
3. The Library modal allows you to view a list of available libraries. Every Library corresponds to a File with Styles and Components that you have access to.
4. Figma will display Components from those Libraries in the Assets Panel.

Full list of libraries (Subject to change)

1. Accessibility

Rules and information on Accessibility.

2. Color Palette

3. Components

Common used molecules

4. Error Pages

e.g. 404 errors

5. Helpers

Tools for annotating your designs

6. Icons

7. Layout

Our Grid, breakpoints, and mobile responsive

8. Live Testing Components

9. Navigation & Chome

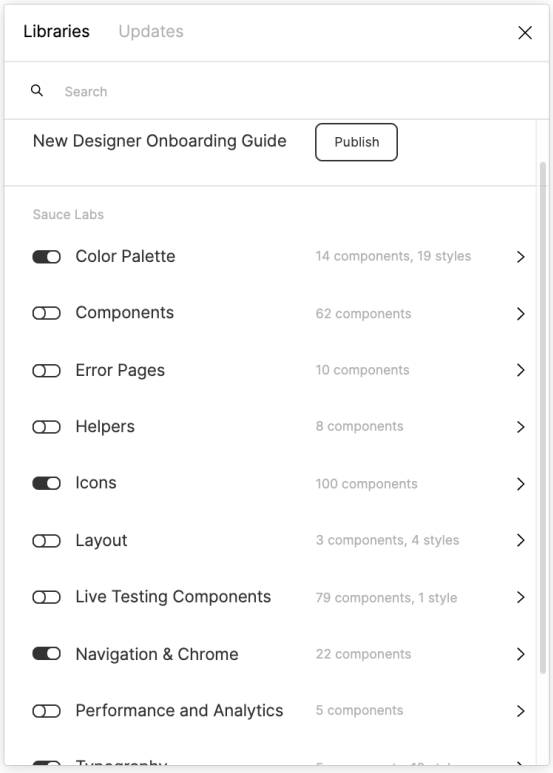
Navigation & Menus

10. Performance & Analytics

11. Typography

12. UI Elements

Form elements, buttons, and other UI molecules.



Team Libraries modal

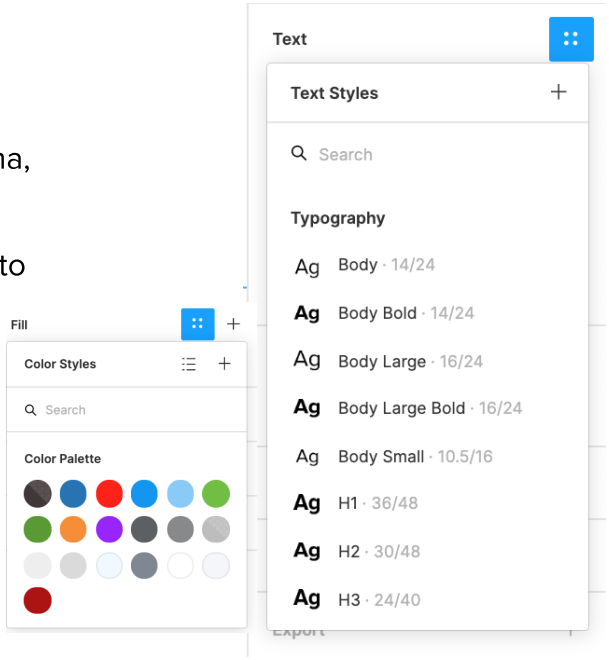
Design System: Type and Color

Using Typography & Color

Type and colors will not show up in the assets panel in Figma, rather, they will show up as “**styles**” in your style menus. These style menus are represented by four small dots next to the tool you would like to adjust (e.g., Fill).

Our current typescale is being worked on. Our main font in app is Proxima Nova.

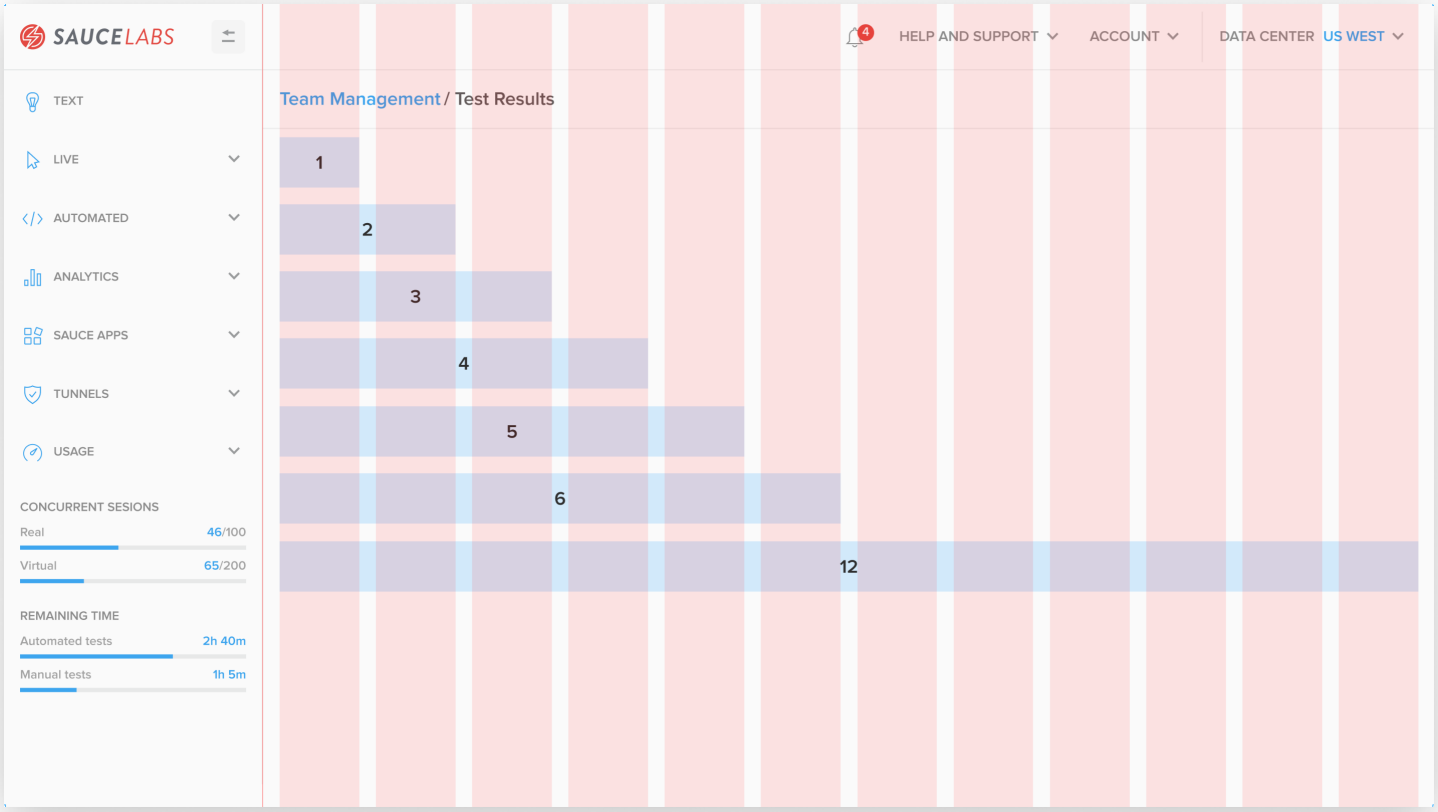
[Download Proxima Nova]



Figma Text and Color Menus

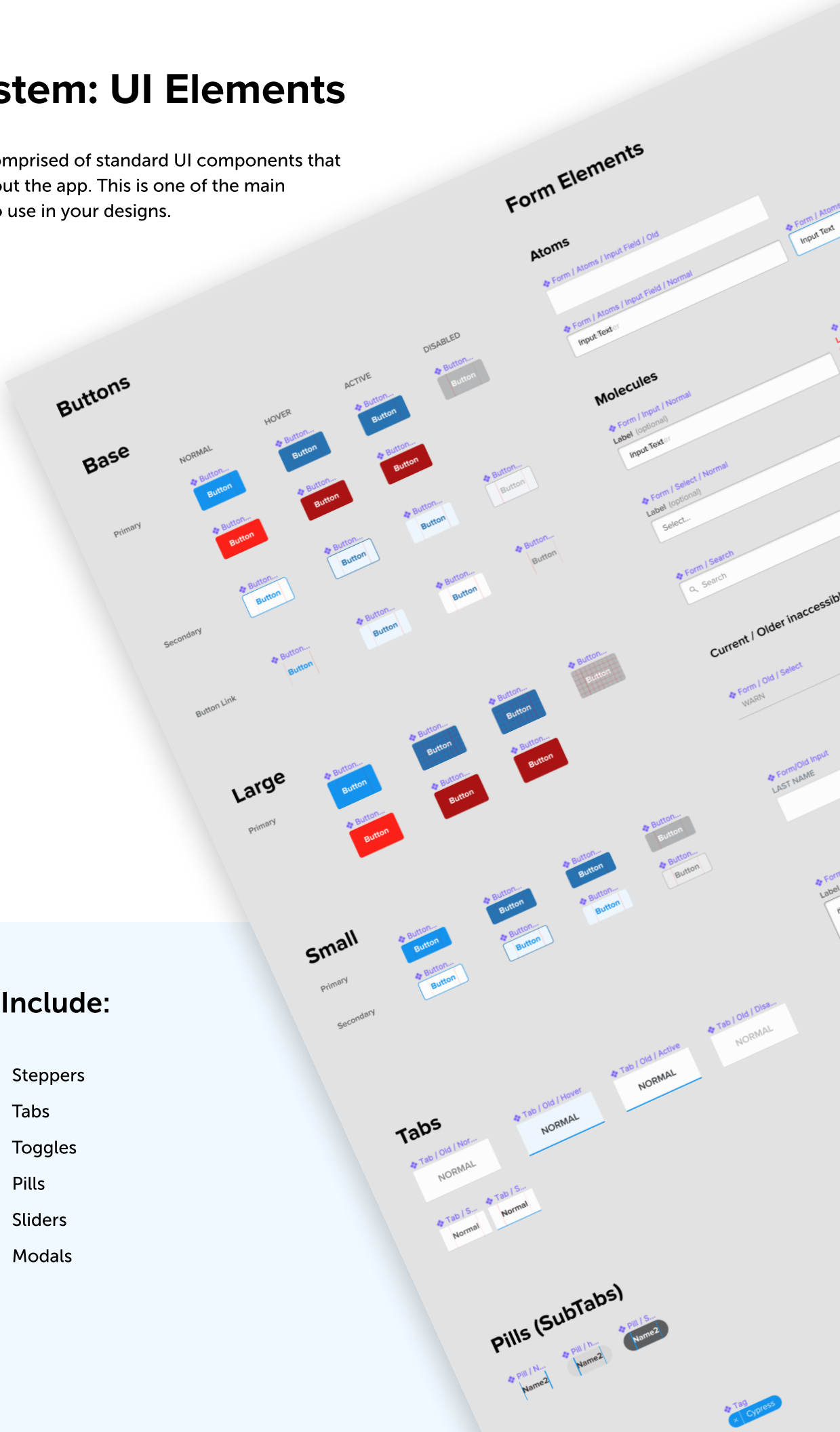
Grid

We are currently using a 12-column layout system, base 8 horizontally, and base 4 vertically.



Design System: UI Elements

Our UI team library is comprised of standard UI components that are often used throughout the app. This is one of the main libraries you will want to use in your designs.

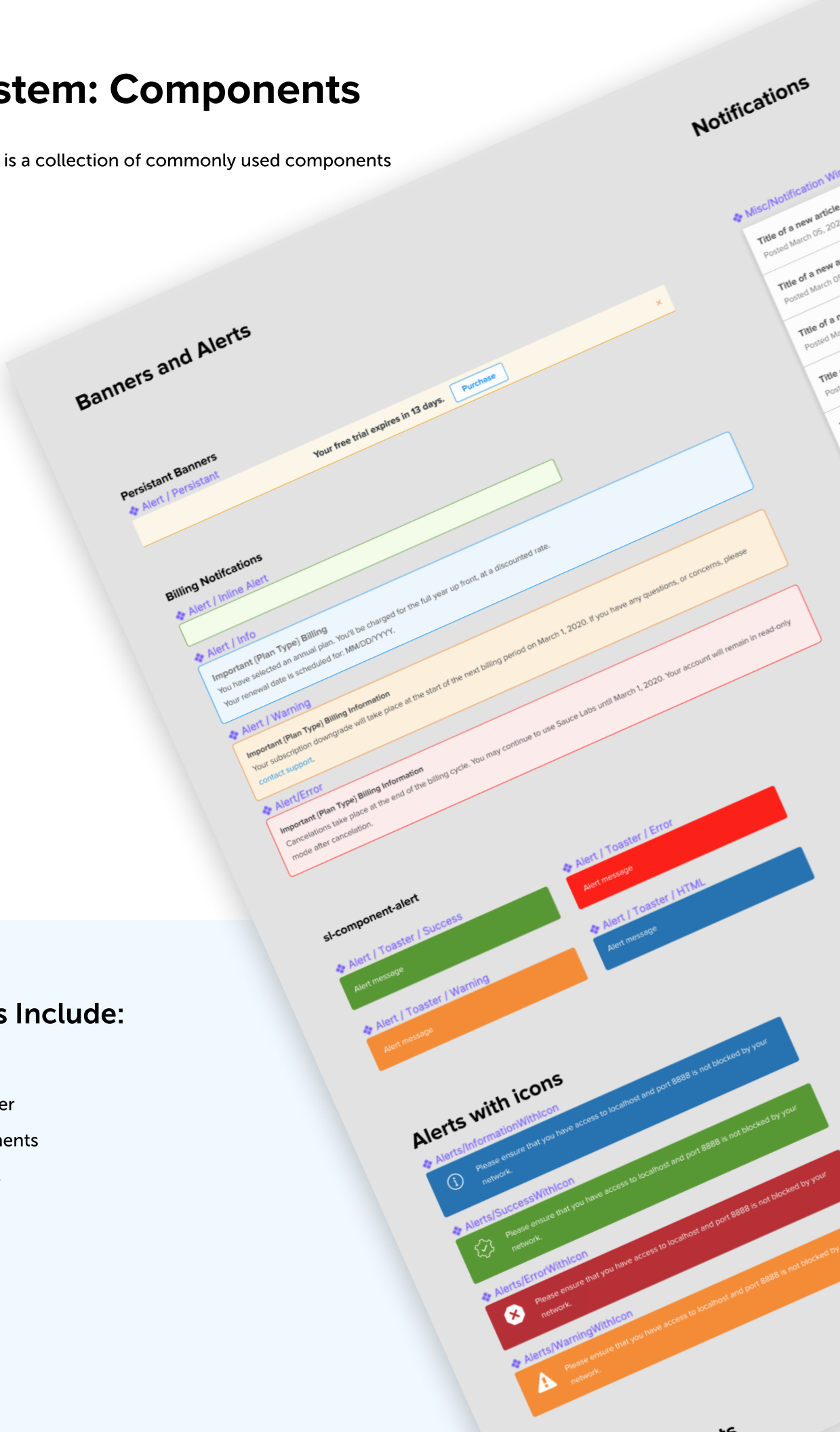


UI Elements Include:

- Buttons
- Cards
- Checkboxes
- Chips
- Form Inputs
- Scroll bars
- Radio Buttons
- Steppers
- Tabs
- Toggles
- Pills
- Sliders
- Modals

Design System: Components

Our components library is a collection of commonly used components within the product.



Components Include:

Alerts

Code Snippets Viewer

Job Details Components

Gauges and Loaders

Notifications

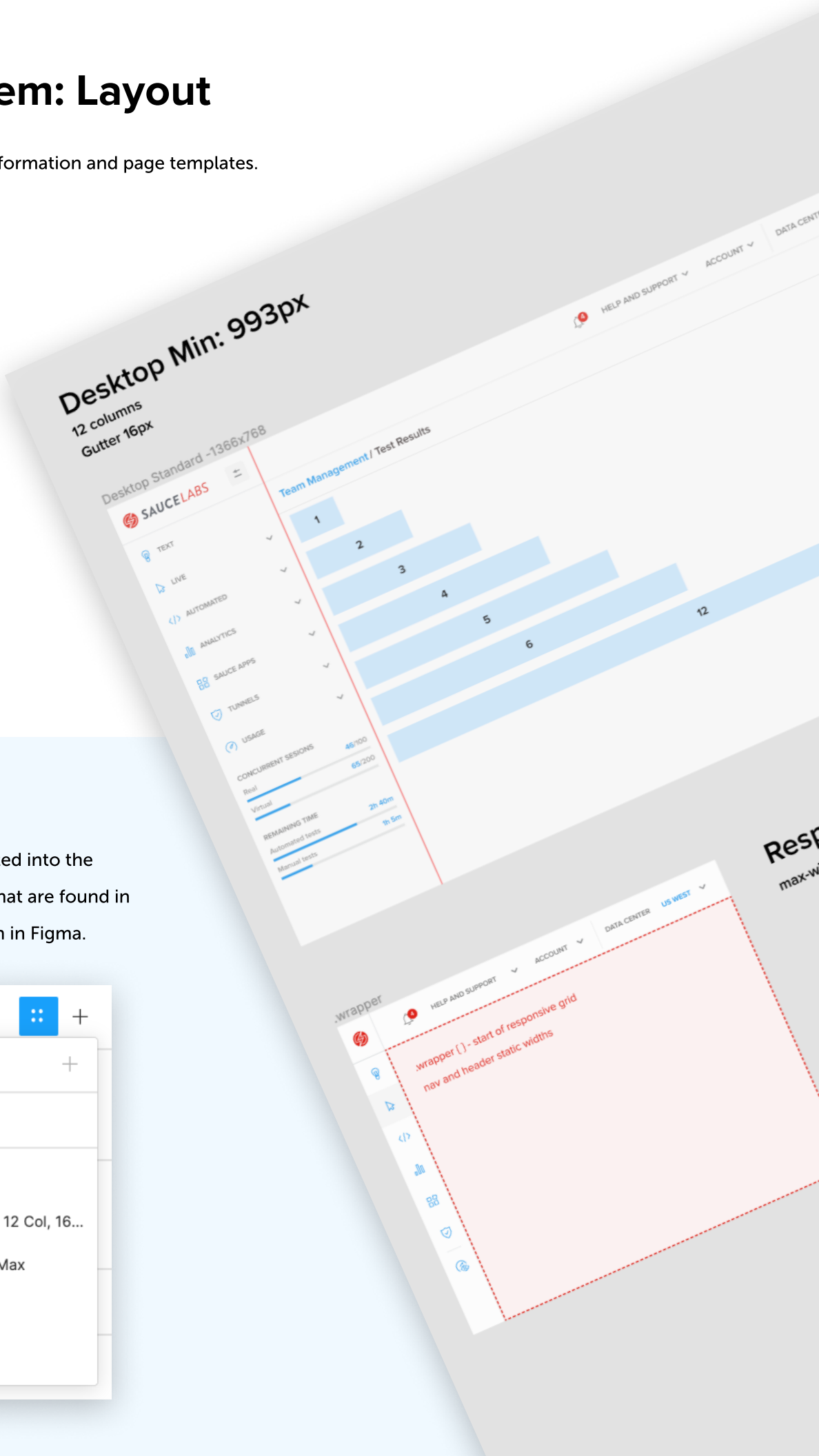
Tooltips

Tables

Misc. Components

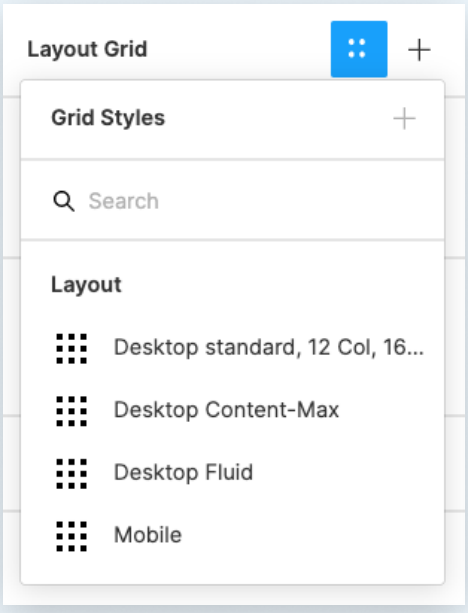
Design System: Layout

This page contains layout information and page templates.



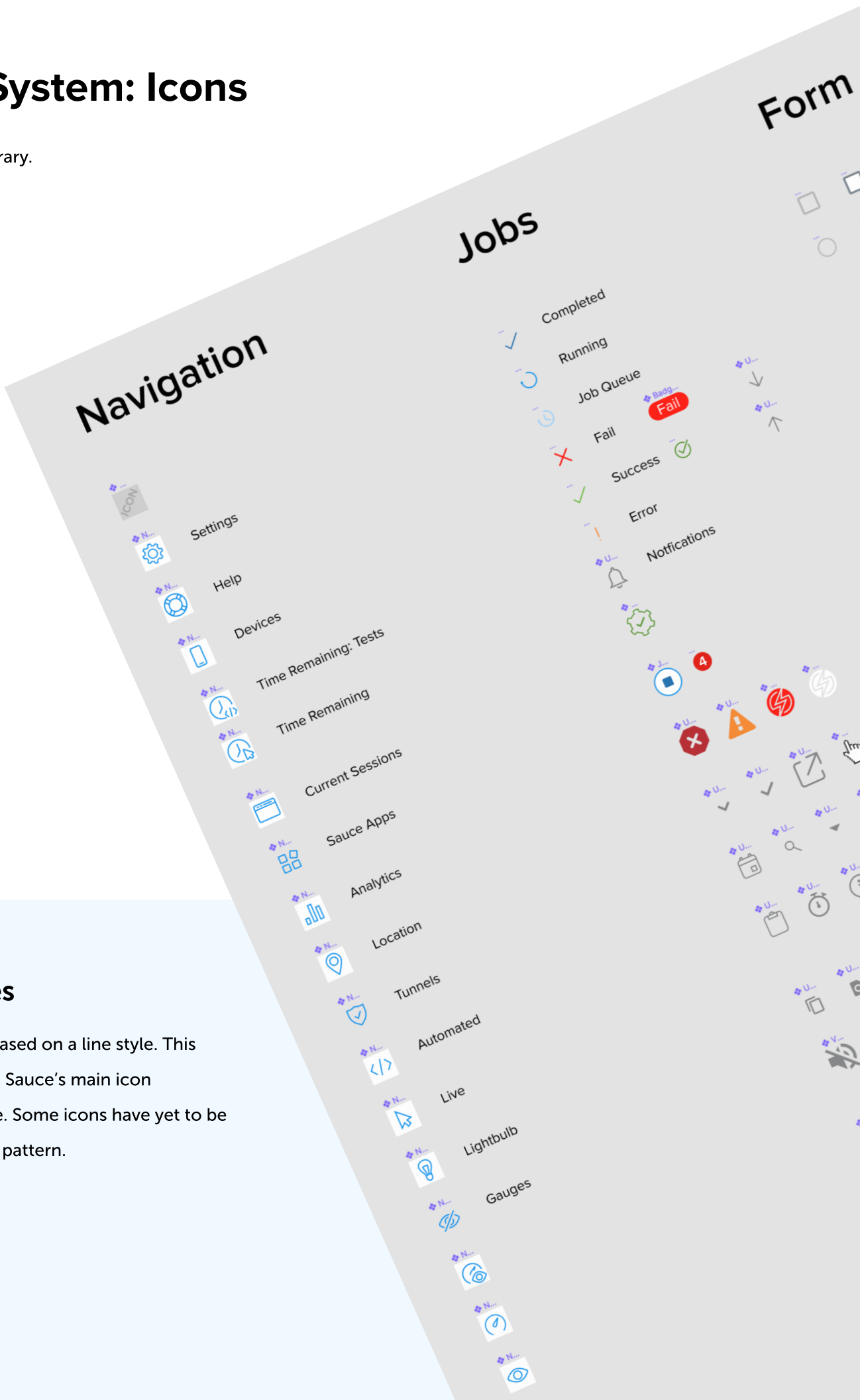
Grid Styles

The layout grid is imported into the document using styles that are found in the “Layout Grid” section in Figma.



Design System: Icons

Our in-app icon library.



Icon styles

Our icons are based on a line style. This falls in line with Sauce's main icon illustration style. Some icons have yet to be folded into this pattern.

Design System: Governance

You can have a comprehensive design system that contains a slew of well-structured components, thorough documentation, thoughtful guidelines, and a well-considered design language. But if a design system user can't get done what they're trying to get done, the whole system risks obsolescence.

According to Brad Frost, design system governance process is one of the most important ingredients of a healthy design system that stands the test of time.

Creating a New Component

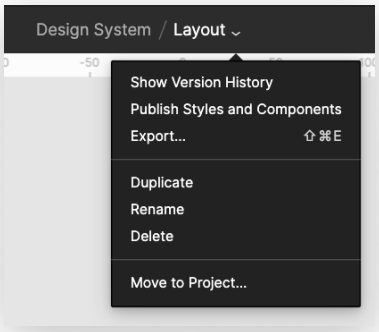
If it's agreed between engineering and design that a new component needs to be created, it's important to determine the whether this is a component that everyone might use in their projects, or if this is a one-off instance.

1) A "**snowflake**", which is a one-off component that only really pertains to one specific product or use case (such as a concurrency calculator, specific data visualizations, or any component that feels especially challenging to abstract into a general-use component).

2) **Part of the design system**, which is a component or variation that is part of library that serves all products.

To create a new component, open the asset library main file. Create your component. From there, you will need to publish your component using the top menu.

Hit "**Publish Styles and Components**" and your component is now officially in the Figma asset team library.



> **Important Note**
> Once you publish a component, it can be accessed by anyone in our team using the asset library.

Editing an Existing Component

Because our assets are independent of the React Component Library, we need to take special precaution adjusting the presentation or functionality of an existing component.

When to Create a New Component?

1. When a component is needed for more than one instance, and can be shared, and resued.

2. There isn't an existing component that could not be used - or extended in a meaningful way.

3. When engineering, and design, both agree it's needed.

Weekly routine

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed.

Mondays: Weekly Standup

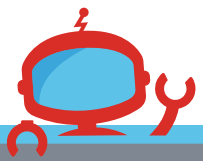
At vero eos et accusamus et iusto odio dignissimos ducimus qui blanditiis praesentium voluptatum deleniti atque corrupti quos dolores et quas.

Wednesdays: Design Review

At vero eos et accusamus et iusto odio dignissimos ducimus qui blanditiis praesentium voluptatum deleniti atque corrupti quos dolores et quas.

Thursdays: Weekly Meeting

At vero eos et accusamus et iusto odio dignissimos ducimus qui blanditiis praesentium voluptatum deleniti atque corrupti quos dolores et quas.



	US (PST)	EU (CEST)
Mondays: Weekly Standup	9:00 AM - 10:00 AM	6:00 PM - 7:00 PM
Wednesdays: Design Review	8:00 AM - 9:00 AM	5:00 PM - 6:00 PM
Thursdays: Weekly Meeting	8:00 AM - 9:00 AM	5:00 PM - 6:00 PM

Tooling

Reporting and Analytics

Mixpanel

Event tracking, customer journey tracking

Looker

Informed data, more overall customer reporting

Google Analytics

Page tracking analysis

Mailchimp

Surveys, customer emails

Productivity

GSuite

Email, spreadsheets, docs, forms (for surveys) and Drive (may your Finder rest in peace)

Zoom

Online meetings

Aha!

Product team backlog and planning

Jira

Tickets

Figma

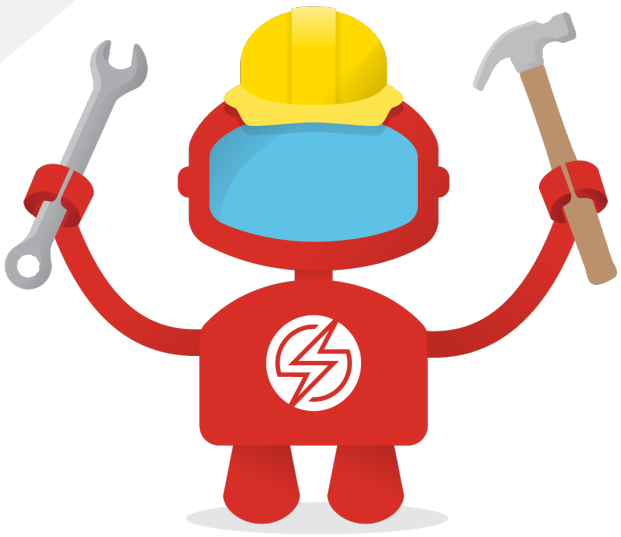
Designer's IDE

Miro

Collaboration

Adobe Creative Cloud

Adobe products & fonts



Product Suite

1. Live Testing

2. Automated Testing

3. Visual Testing

4. Local Testing (Developer Experience)

5. Sauce Connect

6. Performance & Insights (Analytics)

7. Headless

8. Team Management

9. Coming soon...

Testing at the speed of awesome.

Products at Sauce Labs have dedicated cross-functional teams. Some products have cross-team dependencies and can work on different parts of the same app. A typical product team has developers, a Product Manager (PM), a designer, and a scrum master.

Some products are consumer products that we sell. For instance, Live testing. Some products are part of the overall product offering (e.g., Team Management). Some products have yet to be rolled into the final "unified" solution, but efforts are currently underway to unify our product experience.

Live Testing

PM: Victoria Lee

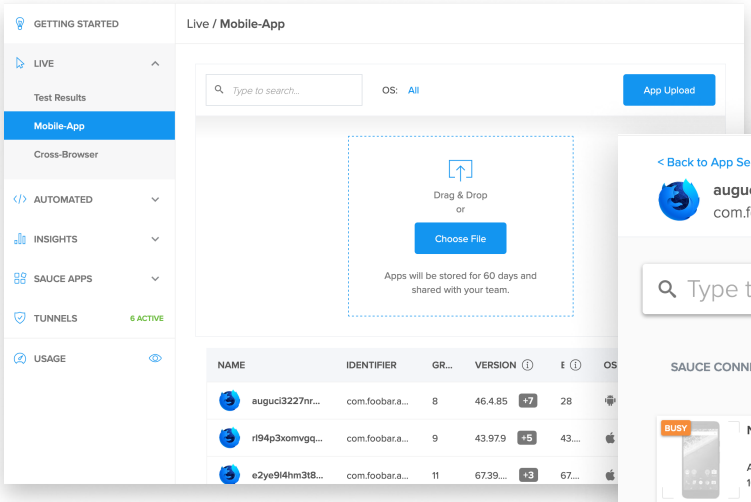
Live testing, often called **manual testing**, is the process of manually testing software for defects. It requires a tester to play the role of an end user whereby they use most of the application's features to ensure correct behavior.

Our Live testing products are broken out into two basic buckets —Mobile App testing, and Cross-Browser testing.

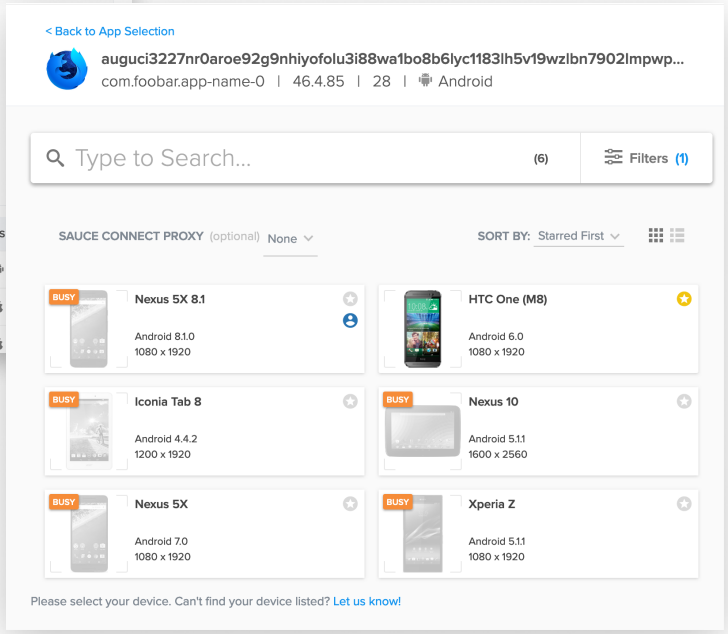
Mobile App Testing

Teams Involved: Live Testing, EMUSIM, RDC

Mobile app testing is the process of testing an application made specifically for iOS and Android devices. It's often referred to as native app testing.



App upload screen.



Device selection screen appears after selecting an app to test.

Cross-Browser Testing

Cross-browser testing is the process of testing a mobile or desktop application in a browser. This kind of testing is performed on a browser in a desktop environment, like the laptop and the browser you are using right now. But cross-browser testing can also performed on a device like a phone or tablet. The difference between testing on a browser on a phone, and testing on the phone itself is the difference in the type of app the user wants to test. A **mobile app** is an Android or iOS application, often referred to as a **native app**. A browser-based app is a website or web-based application that runs on a device. The average user will not know whether the application they are using is native or browser-based.

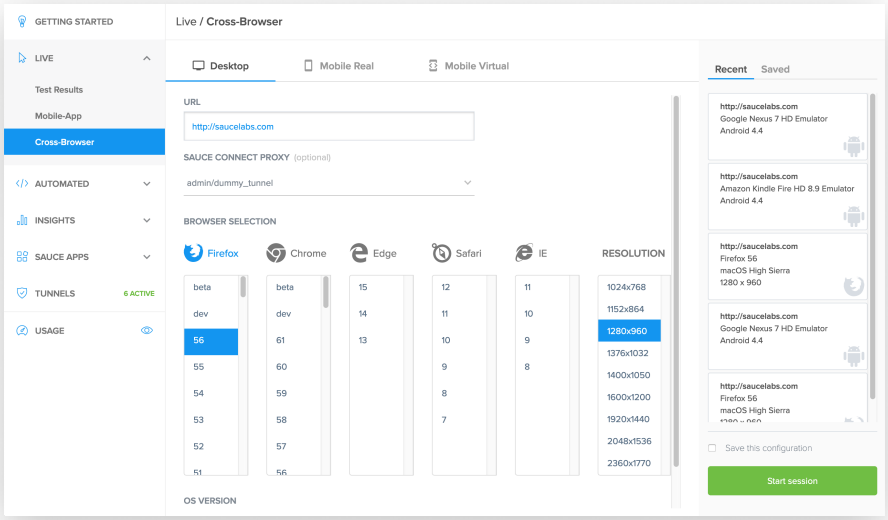
Live Testing

PM: Victoria Lee

Live testing on Sauce is broken out into 3 segments: Desktop, Mobile Real, and Mobile Virtual.

Desktop

Browsers, and operating system VM's. **View the demo here**



This part of our product is set up for our **QA persona** who is doing manual testing—that is, not an automated test, but a test someone has to manually perform.

However, **Developers** often use desktop for specific use cases they wish to manually observe.

Web designers and front-end developers will use this tool to make sure that the presentation layer (what you see) renders correctly accross the internet's most popular browsers.

URL
The website's URL you wish to test.

SAUCE CONNECT PROXY
This is for testing on a local URL - like http://localhost:8080. This is represented in our UI as Tunnels, and is the secure connection between the customer's computer, and Sauce Labs infrastructure.

BROWSER SELECTION
This is where the customer selects the single browser they wish to test and the resolution they wish to test on.

OS VERSION
Sets the operating system you wish the browser to be run on.

RECENT
This is the last few configurations they have manually set up. Often times QA will test across common scenarios, browsers, and devices. This list provides the customer with the ability to quickly fire up a configuration.

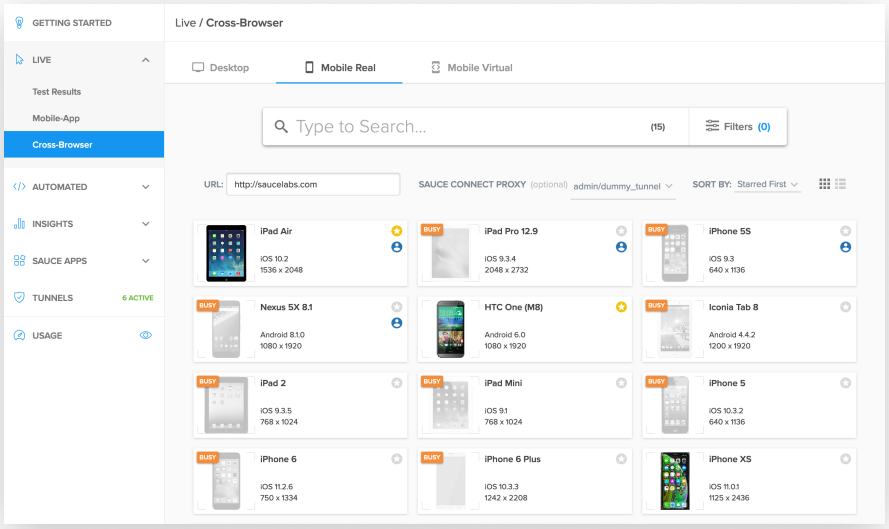
SAVED
Additionally, if the customer wishes to save their configuration for use later, they have the option to save the configuration and access it again from the Saved side bar item.

Live Testing

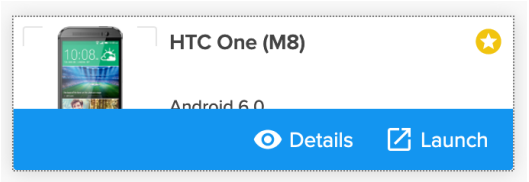
Mobile Real - AKA Real Device Cloud

PM: Victoria Lee (Live Testing), Karla Schoenicke (RDC)

Our real device cloud (RDC) consists of actual phones and tablets plugged into a rack in a data center. Sauce Labs has a huge selection of devices for customers to choose from.



Running a Mobile Real tests consists of hovering over a phone from our device catalog and selecting Launch.



Additionally, you may see the details of the phone.

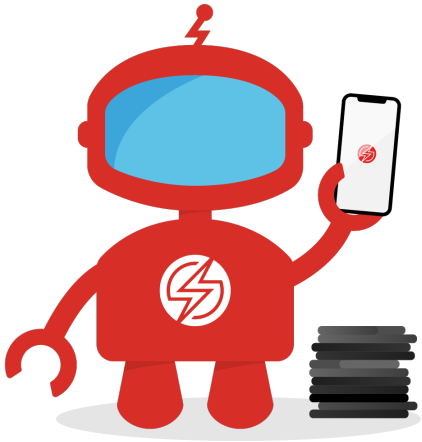
Fun Fact: This portion of the Sauce Labs product offering comes from an acquisition in Berlin called Test Object.

TYPE TO SEARCH
Allows you to type ahead search, and filter the device catalog to find exactly the device and specifications you want to run.

URL
The website's URL you wish to test.

SAUCE CONNECT PROXY
This is for testing on a local URL - like http://localhost:8080. This is represented in our UI as Tunnels, and is the secure connection between the customers computer, and Sauce Labs infastructure.

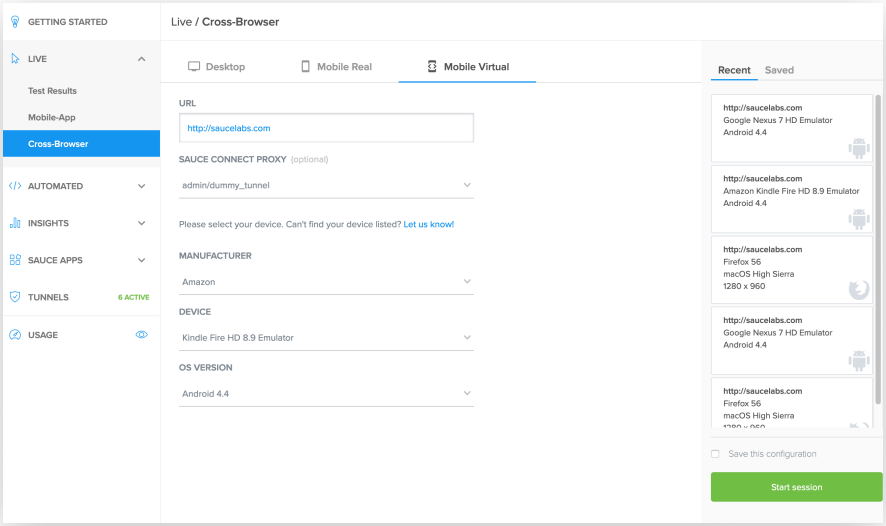
DEVICE CATALOG
An informal name given to the list of devices presented for use.



Live Testing

Mobile Virtual - AKA EMUSIM

Virtual devices running in virtual machines.



Our virtual mobile offering is our collection of every emulator and simulator we have.

Read more:

[Simulators vs. Emulators: What's the Difference, Anyway.]

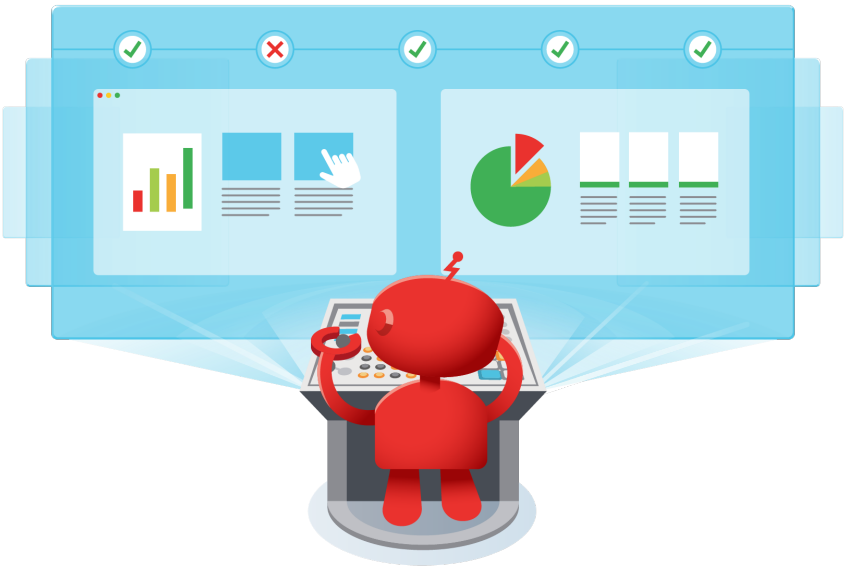
URL
The website's URL you wish to test.

SAUCE CONNECT PROXY
This is for testing on a local URL - like http://localhost:8080. This is represented in our UI as Tunnels, and is the secure connection between the customer's computer, and Sauce Labs infrastructure.

MANUFACTURER
Brand of device.

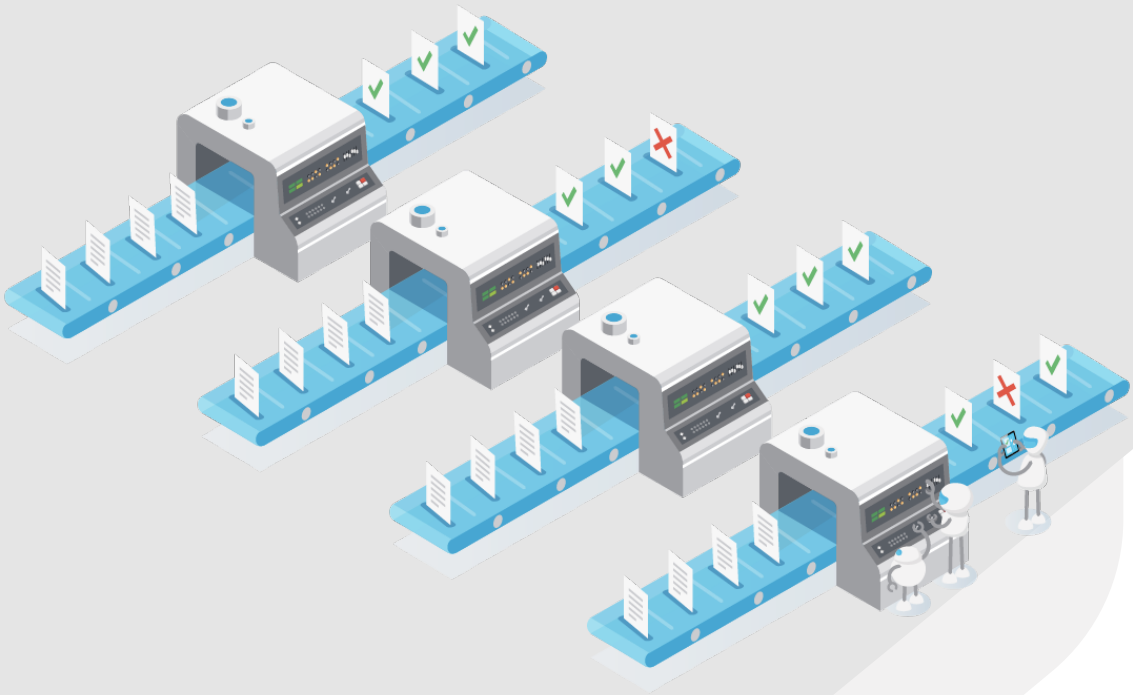
DEVICE
The type of emulated or simulated device you want to run.

OS VERSION
The operating system for the virtual device.



Automated Testing

In software testing, test automation is the use of software separate from the software being tested to control the execution of tests and the comparison of actual outcomes with predicted outcomes. Test automation is critical for continuous delivery and continuous testing.
Watch it work



CI/CD

In software engineering, CI/CD or CICD generally refers to the combined practices of continuous integration and either continuous delivery or continuous deployment. CI/CD bridges the gaps between development and operation activities and teams by enforcing automation in building, testing, and deployment of applications.

Selenium

As the co-developers of Selenium, our team of experts maintains the most comprehensive and reliable platform for web testing. Focus on automating your tests to accelerate releases, and leave the infrastructure maintenance to Sauce!

Appium

Sauce Labs has the experts for mobile test automation using Appium, the open source test automation tool for testing mobile applications. Our team contributes to the Appium community and provides the expertise for organizations that need more help with building their test automation strategy.

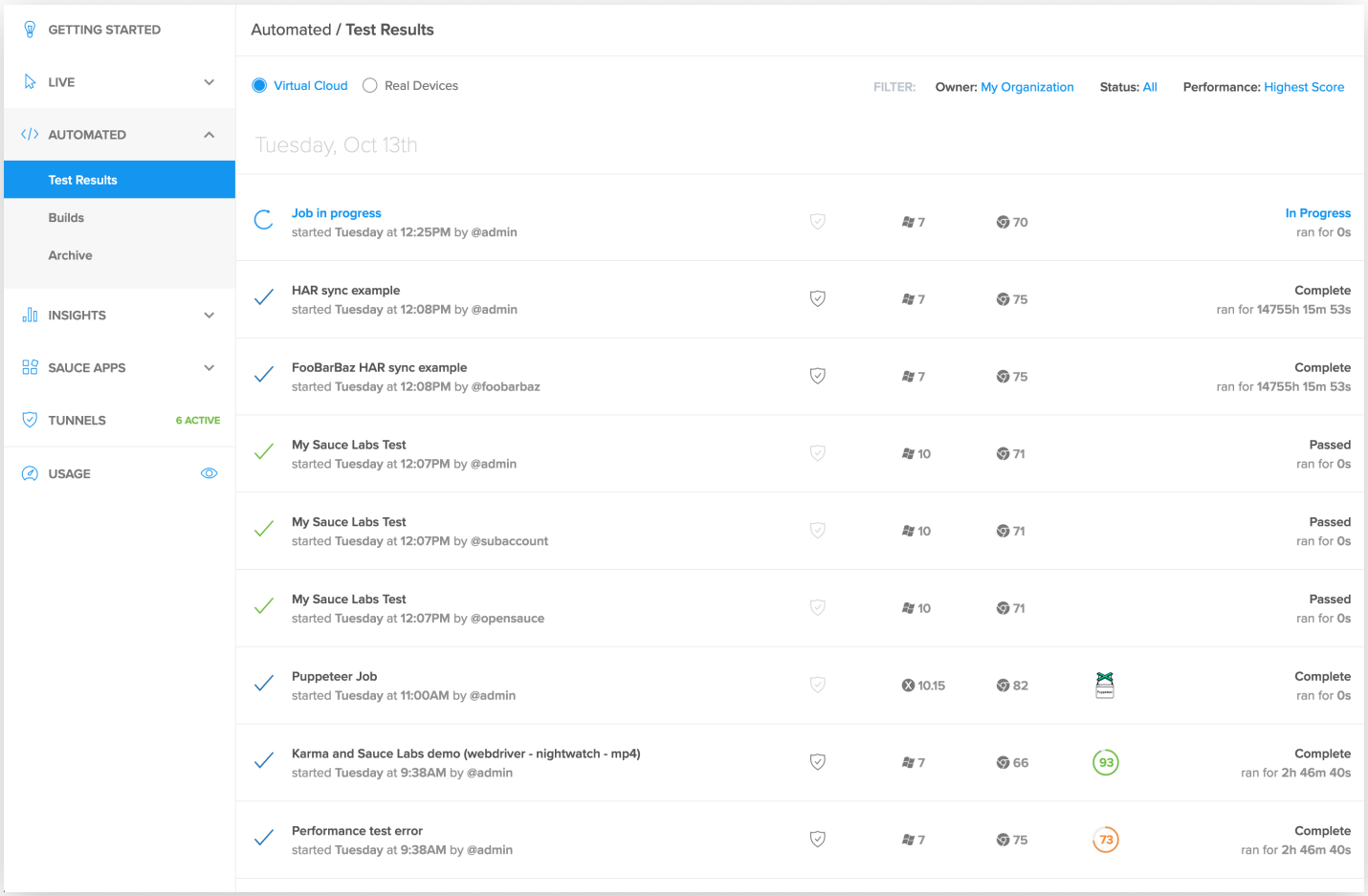
Other Frameworks

Additionally, we support several other frameworks, including Playwrite, Puppeteer, Jest, Cypress, and more.

Automated Testing

Test Results Page

The test results page houses the most recent tests that have been conducted on Sauce. These tests appear in this list. Clicking on any test will give you a detailed look at how the test performed.

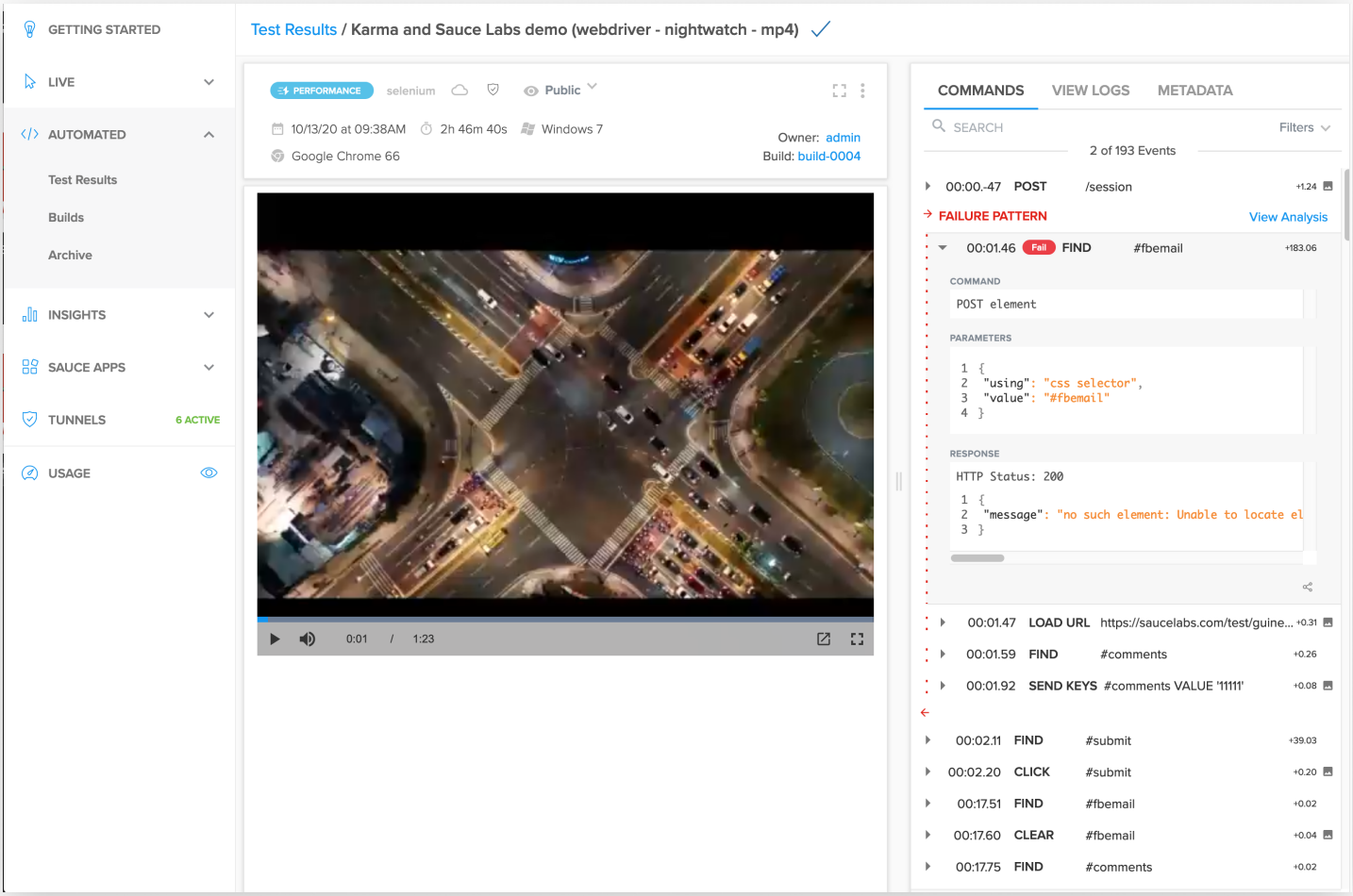


Automated Testing

Job (Test) Details Page

The test details page, sometimes called Job Details page, is one of the most important pages in our UI. This page is dedicated to helping the user debug their automated tests.

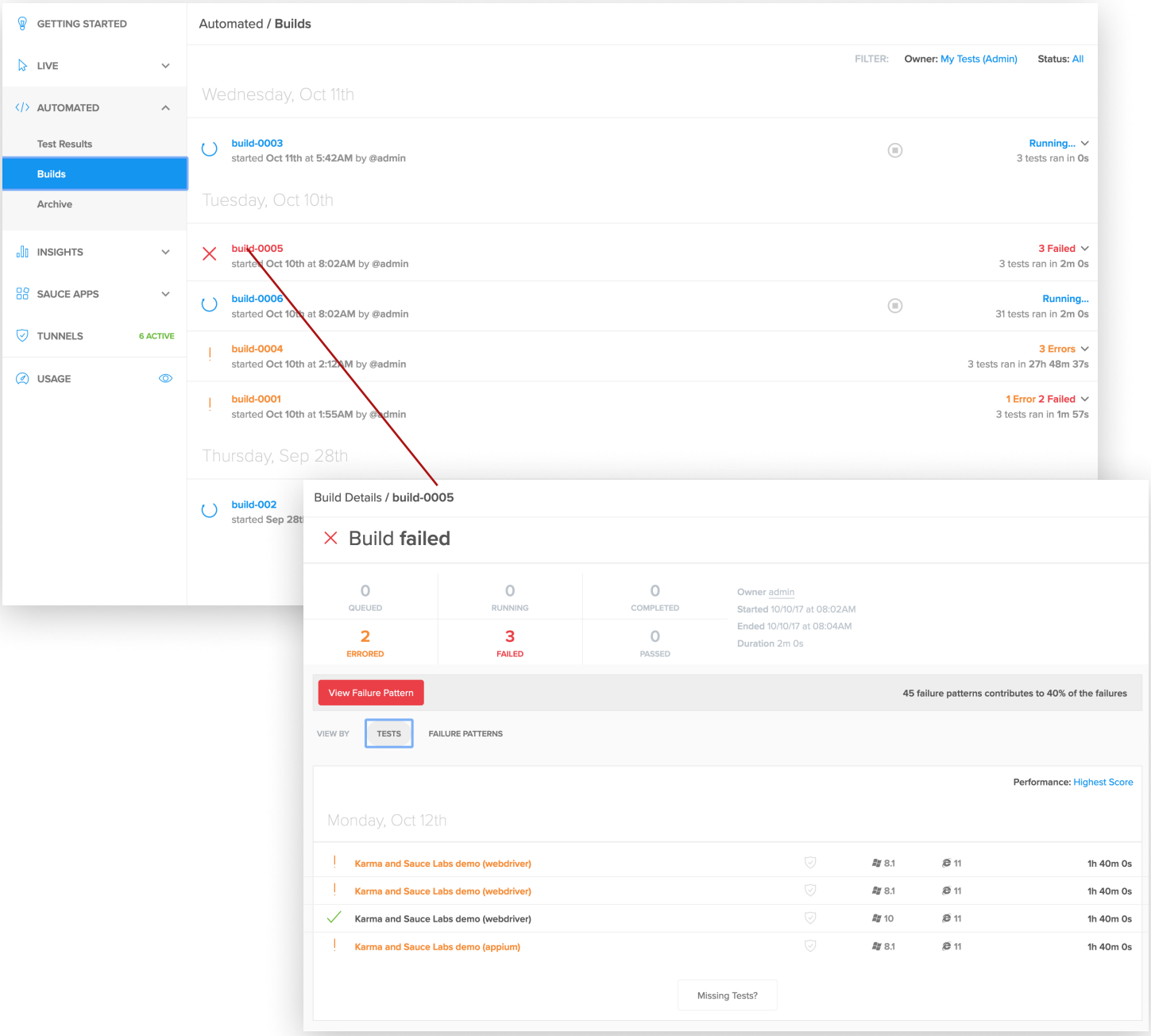
Top use cases for this screen include:
1. Watch a video of the test performed.
2. Evaluate failed commands, and watch the video of the script failing.
3. View the logs.
4. View, or download any screenshots of the failed tests or failed commands.
5. View and download additional meta about the test.
6. Set visibility of the test - often times the URL to this test is copied and pasted into troubleshooting tickets.



Important Note
Most of our customers come to this page directly via a URL that has been generated by their pipeline tools (e.g. Jenkins).

Builds

Builds are simply a **collection of tests**. Our customers can configure their tests attributes to run in collections.



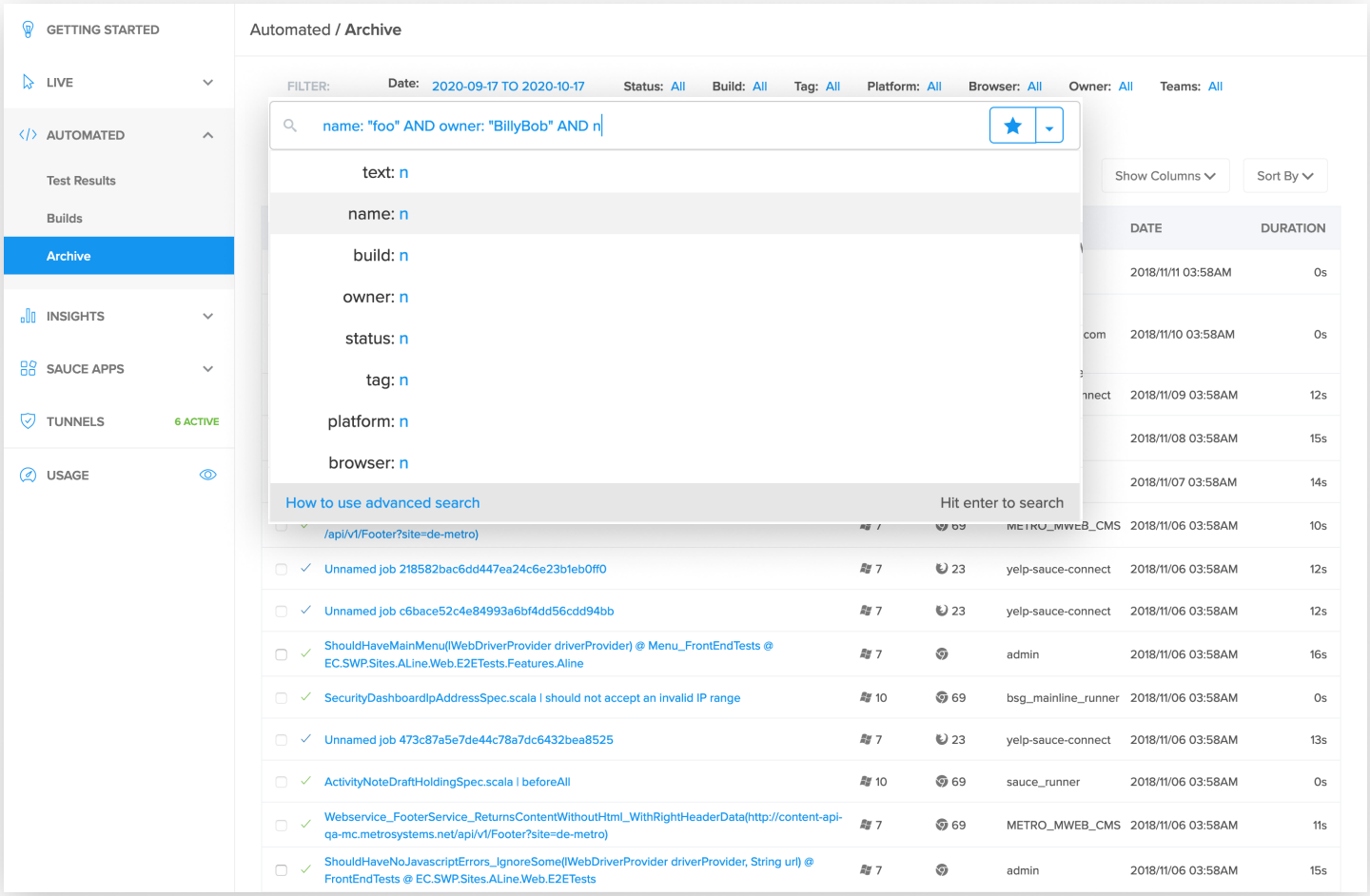
Important Note

Only about 20% of our customers are using builds, but those that do, find the feature very important. Builds may also be the wrong term for this, but it just stuck. Both Visual testing and Automated testing support Builds as part of their test organization.

Archives

The Sauce Labs Archives page provides a handy way to manage historical information about your tests and builds. The Archives page contains all the tests and builds you have run since you opened your Sauce account, and new tests and builds are automatically added to it, while the Dashboard only displays the the last 50 builds or the last 25 tests you have run.

Important Note
We only keep assets such as screenshots, logs, and videos for 30 days. Currently, this is the only place where one of our customers can go in and search for their tests.



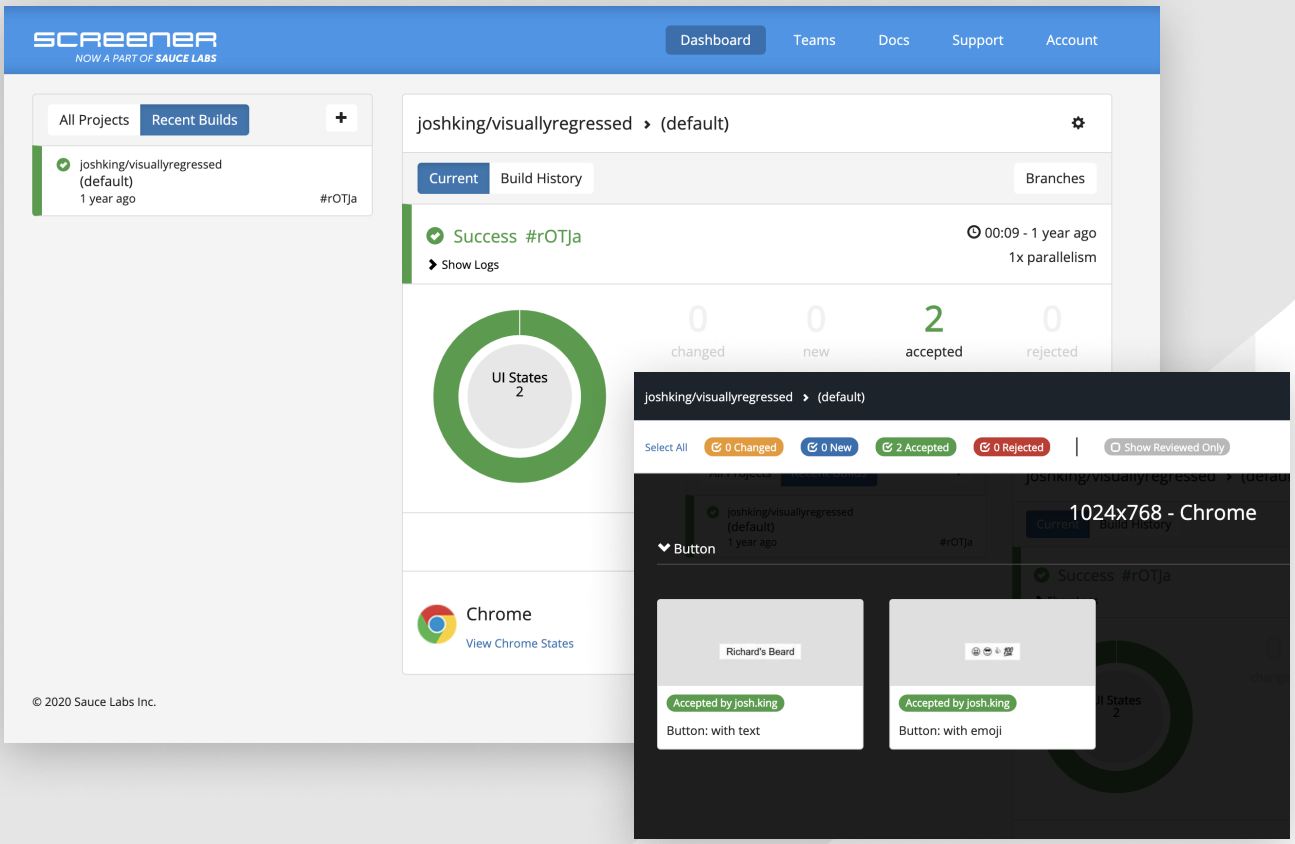
Search & Filters
You can use any of these filters singly or in combination to search through the tests and builds on your Archives page. The Example column shows how you could construct a search using a specific filter in the Search text field. See Searching for Test Results and Builds on Your Archive Page for examples of how to build structured searches using multiple filters in the Search field.

Further, custom searches can be saved and accessed again later.

Visual Testing

Visual testing, also known as user interface (UI) testing in software development is how developers ensure that a web or mobile application they are building appears to the end user as it was originally intended.

Our Visual Testing product was an acquisition called Screener. Screener breaks out the two main testing features in 2 segments: Componets, and E2E (or end-to-end) testing.



End to End (E2E) Testing

End-to-end testing is the process of testing your entire system from start to finish. The purpose of this is to ensure that information is passed between components within the system in the correct manner, and that the integrity of the information is maintained through the process.

Component Testing

Sauce Visual Component Testing provides organizations the ability to quickly automate the testing of their user interface (UI) components across multiple browsers and screen resolutions. This improves developer productivity by automatically detecting visual regressions early in the development lifecycle.

Component testing with Visual is set up for Storybook.

Headless

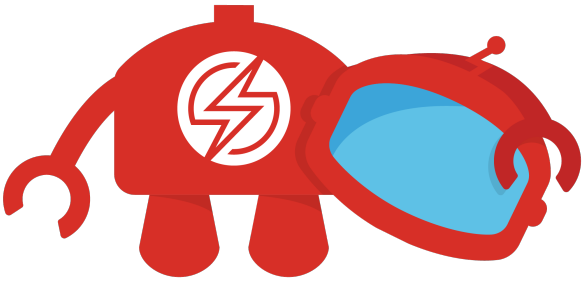
What is headless?

Sauce Headless provides an efficient solution to test against headless Chrome and Firefox browsers, giving developers fast pass/fail data on early pipeline component tests, sanity checks, and pull request tests.

“Headless browser” is a funny-sounding term that refers to something that software test engineers use all the time: a web browser that is not configured with a Graphical User Interface.

A headless browser is a type of software that can access webpages but does not show them to the user and can pipe the content of the webpages to another program. Unlike a normal browser, nothing will appear on the screen when you start up a headless browser, since the programs run in the backend.

Read more



Insights (Analytics)

Insights provides data for engineers and leaders alike to understand how their tests are performing over time, allowing them to quickly identify and remediate risk, improve productivity, and create digital confidence in their entire organization.

Trends

The trend visualizations can provide you with a quick overview of what's going on with your tests as a whole, and applying filters to the visualizations enables you to dig into the data to generate answers to specific questions about test performance. This topic will show you how to use filters to answer a common question: how well does your site under test perform with a new version of a browser, as compared with an earlier version of that browser on the same operating system, and how well do your tests for these browsers perform?

Insights

Sauce Insights provides real-time visibility into your testing efforts. With customizable reports to help you quickly identify bottlenecks and high impact issues, along with actionable insights into how you can improve efficiency, Insights helps identify what to fix first so you can build digital confidence and release better software faster.

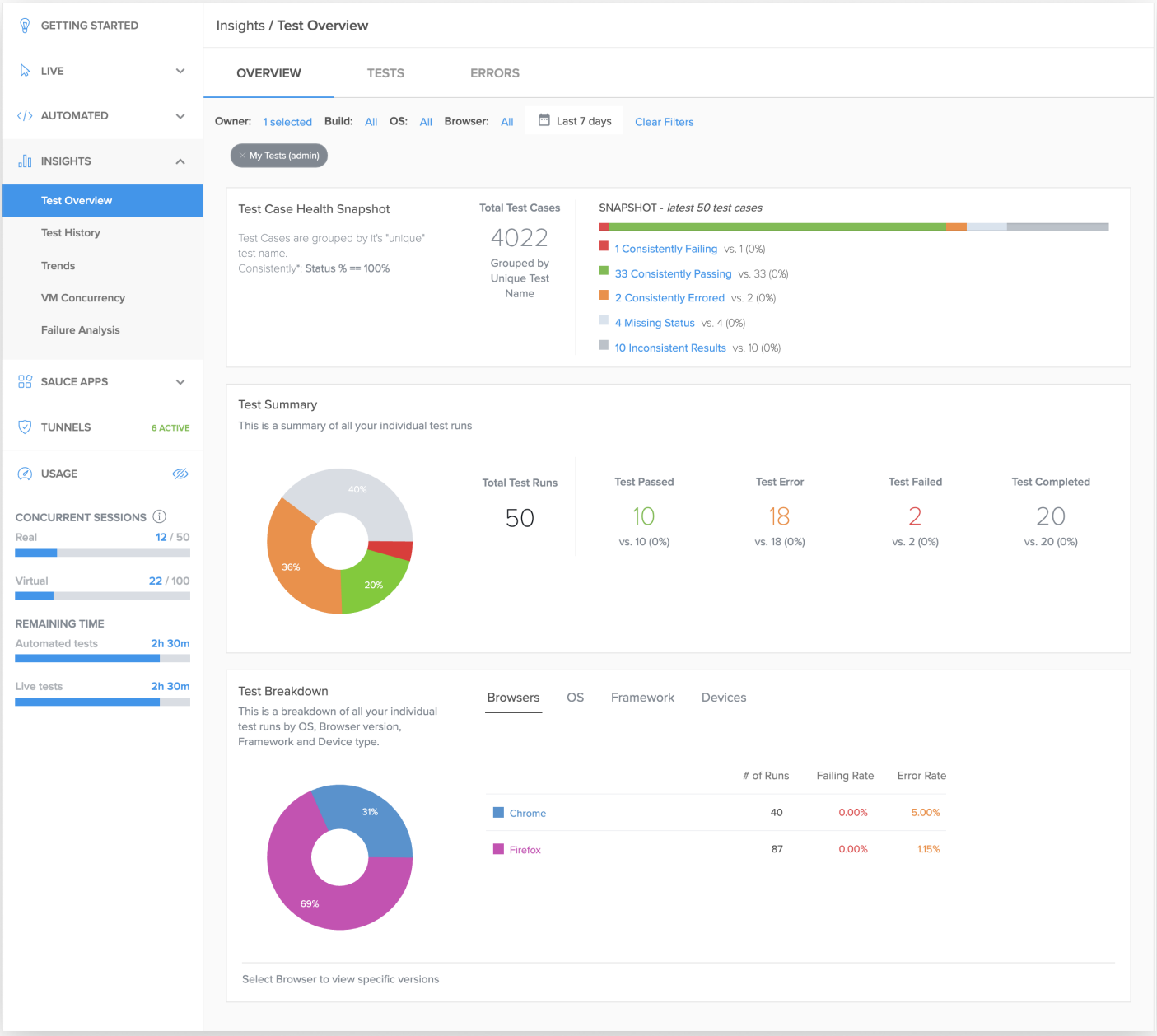
Performance

Sauce Performance provides a single solution for both front-end functional and front-end performance metrics using Selenium test automation scripts, eliminating multi-tool complexity and improving developer productivity.

Sauce Performance helps developer and QA organizations enhance and maintain the user experience of their apps with detailed front-end performance metrics and the ability to capture and address regressions early in the delivery pipeline as part of CI/CD. Eliminate costly delays without having to create new scripts or deploy new infrastructure.

Test Overview

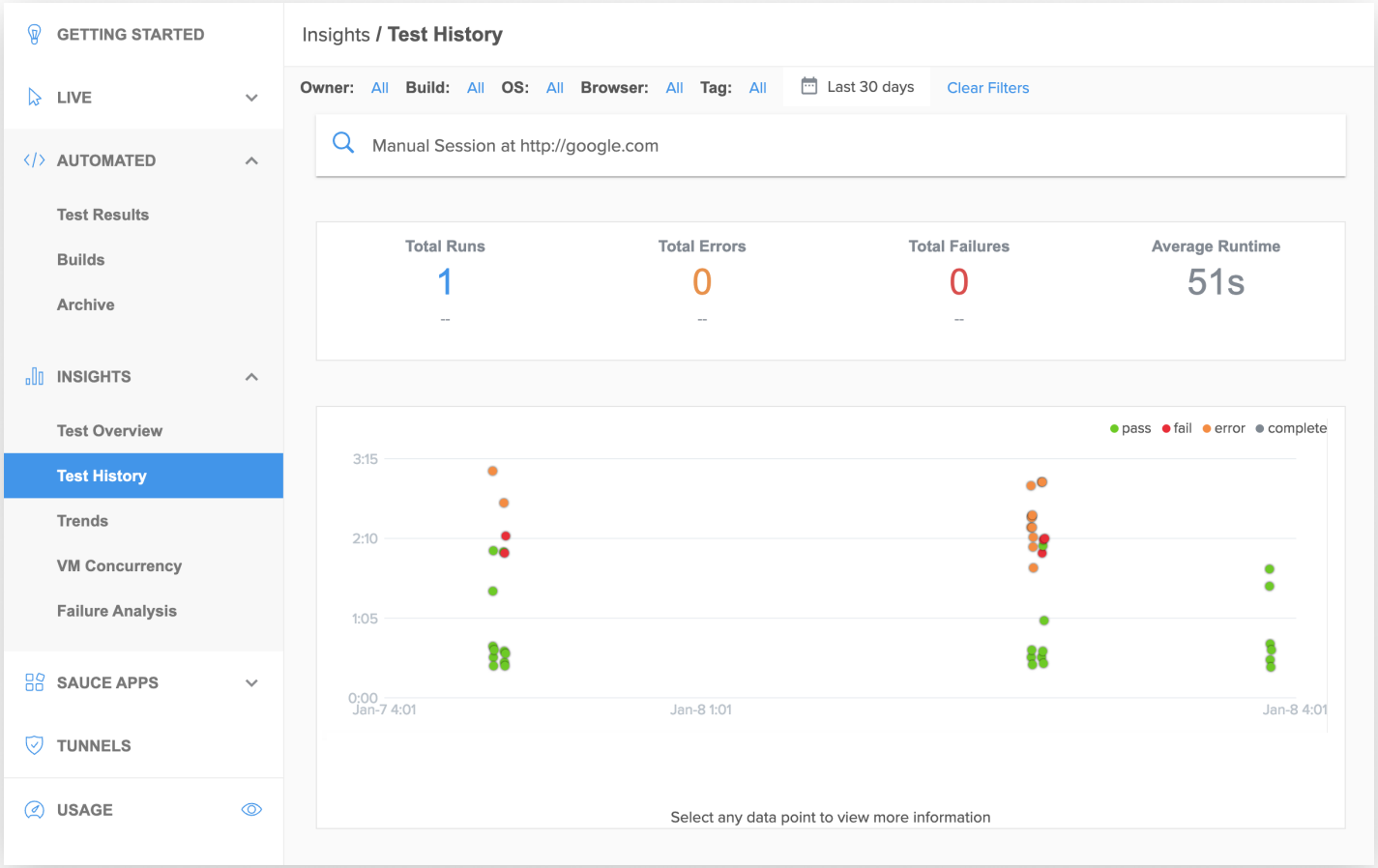
The test results page houses the most recent tests that have been conducted on Sauce. These tests appear in this list. Clicking on any test will give you a detailed look at how the test performed.



Test History

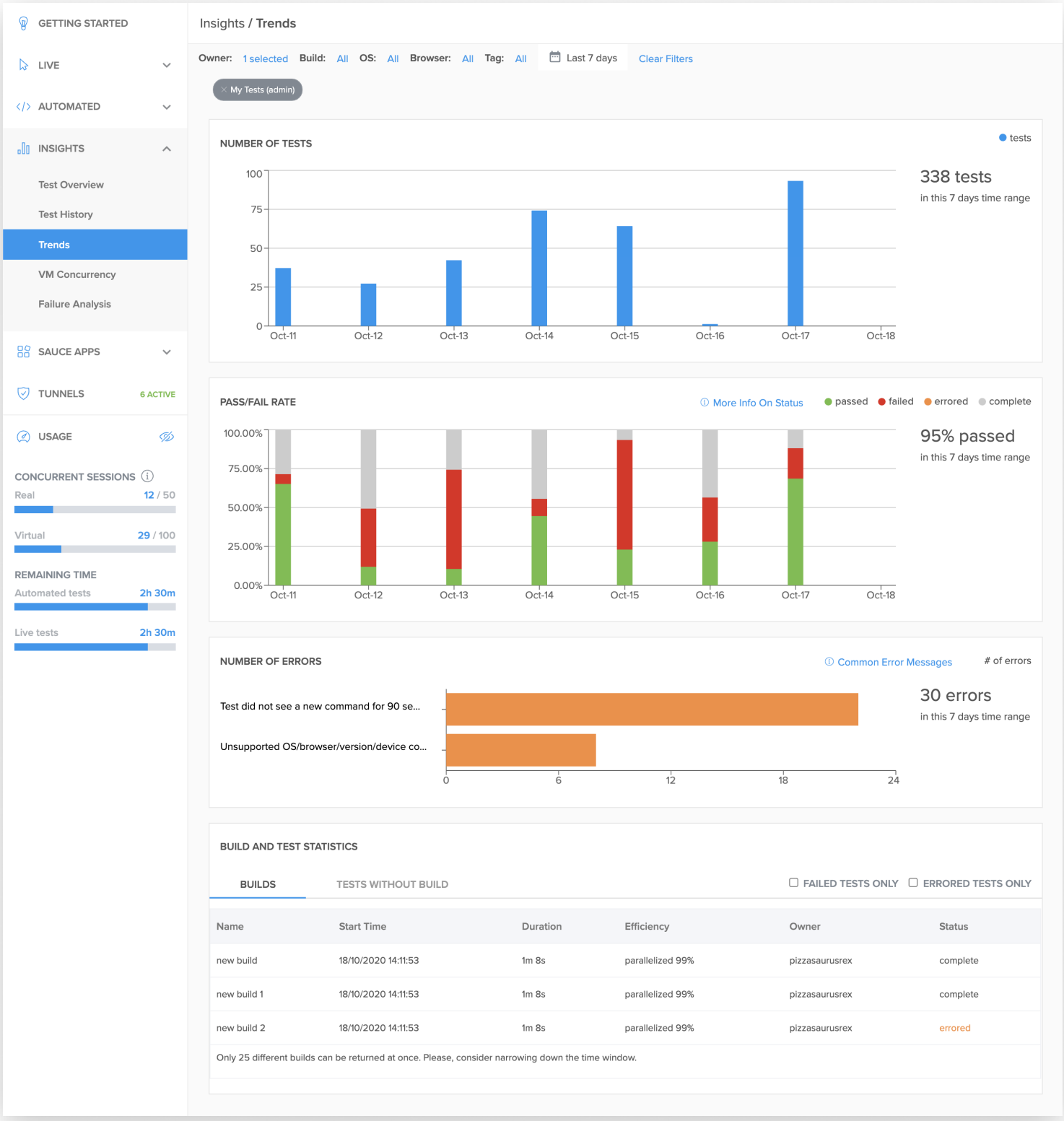
The Test History page gives another historical look on tests.

read more



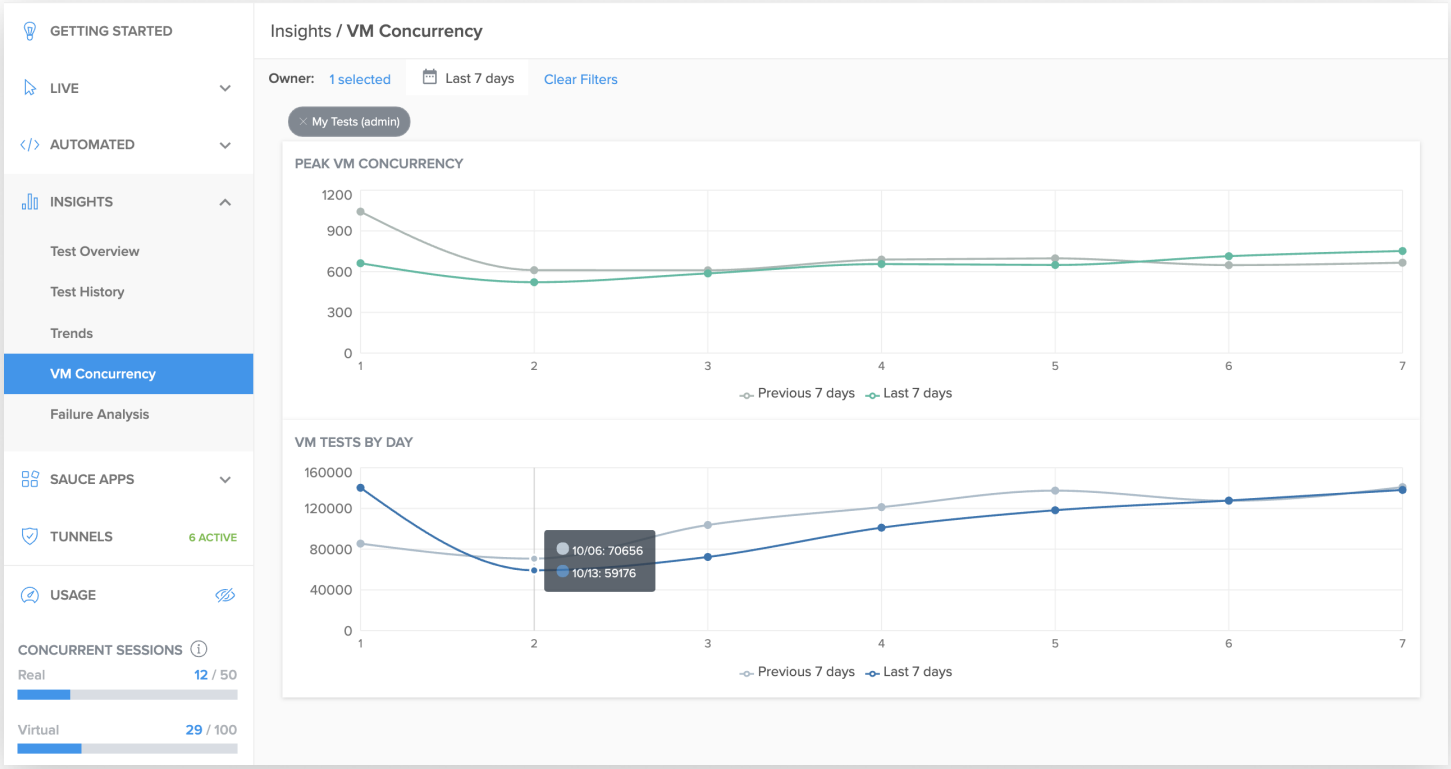
Trends

The Trends visualizations gives our customers information on their tests over time.



VM Concurrency

This view provides our customers with a look on how their VMs (desktop and EMUSIM) concurrency is being spent within the system. Real device concurrency reporting has yet to be aded.



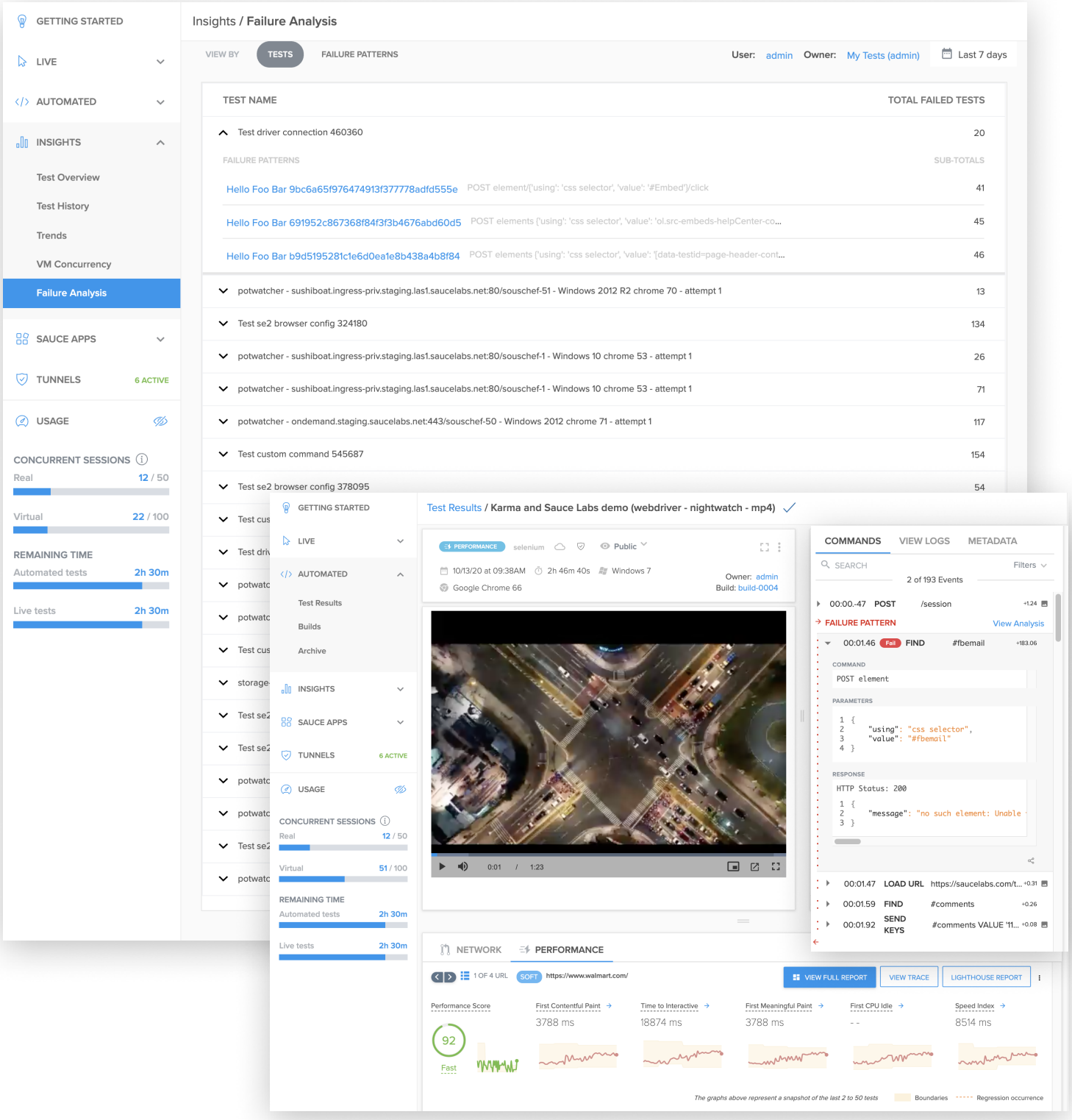
Important Note

Concurrency, or parallel tests, is managed as teams in Team Management.

Failure Analysis

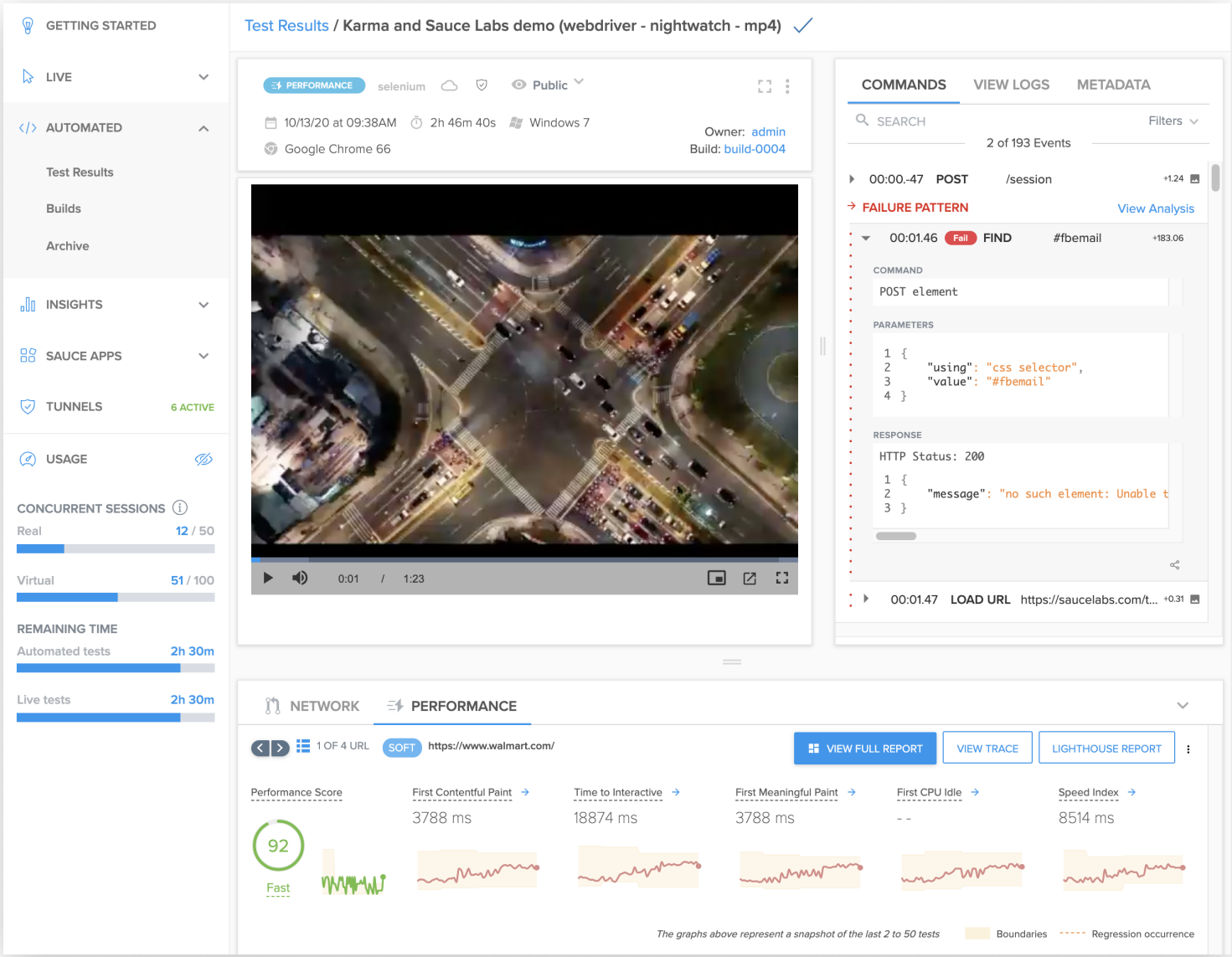
Failure Analysis gives you insight into where you can immediately improve your test execution on Sauce Labs. With a single dashboard view within the Sauce UI, Failure Analysis uses machine learning to view pass/fail data and Selenium and Appium command logs to unearth common failures, and how they are affecting the test suite as a whole. It then presents a report with tabs that aggregate patterns that are indicative of failure.

read more



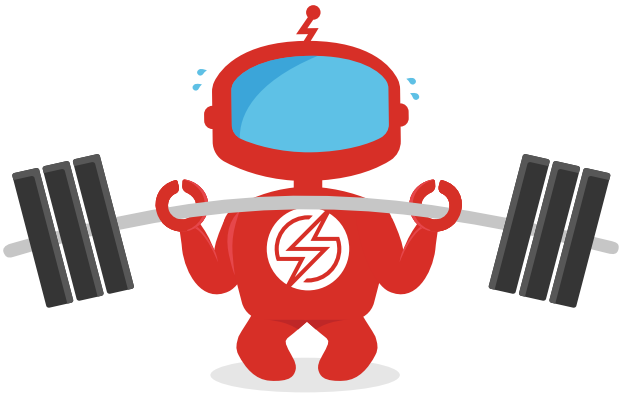
Performance

Front-end performance testing is a set of tools that complement your functional testing strategy with metrics related to the speed and reliability of your UI rendering. Incorporating performance testing allows you to ensure consistency across multiple environments, detect performance degradation early in the development cycle, and implement key changes that yield a faster, richer experience for your users.

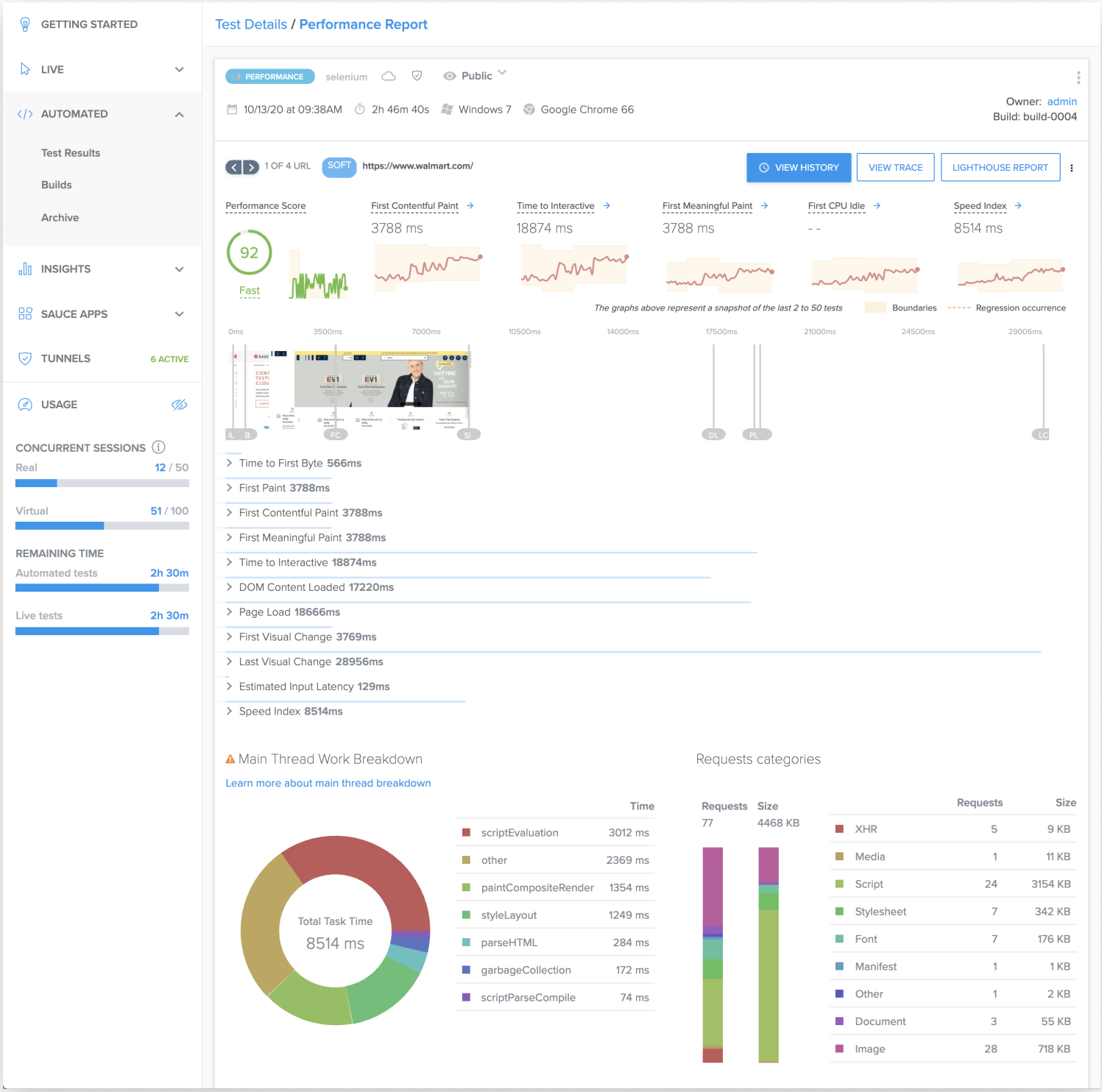


At the moment, Performance is only available for websites / web testing, not for native apps.

Our performance products integrate into our test debugging environment in the Test Details page in the bottom section under Network and Performance tabs.



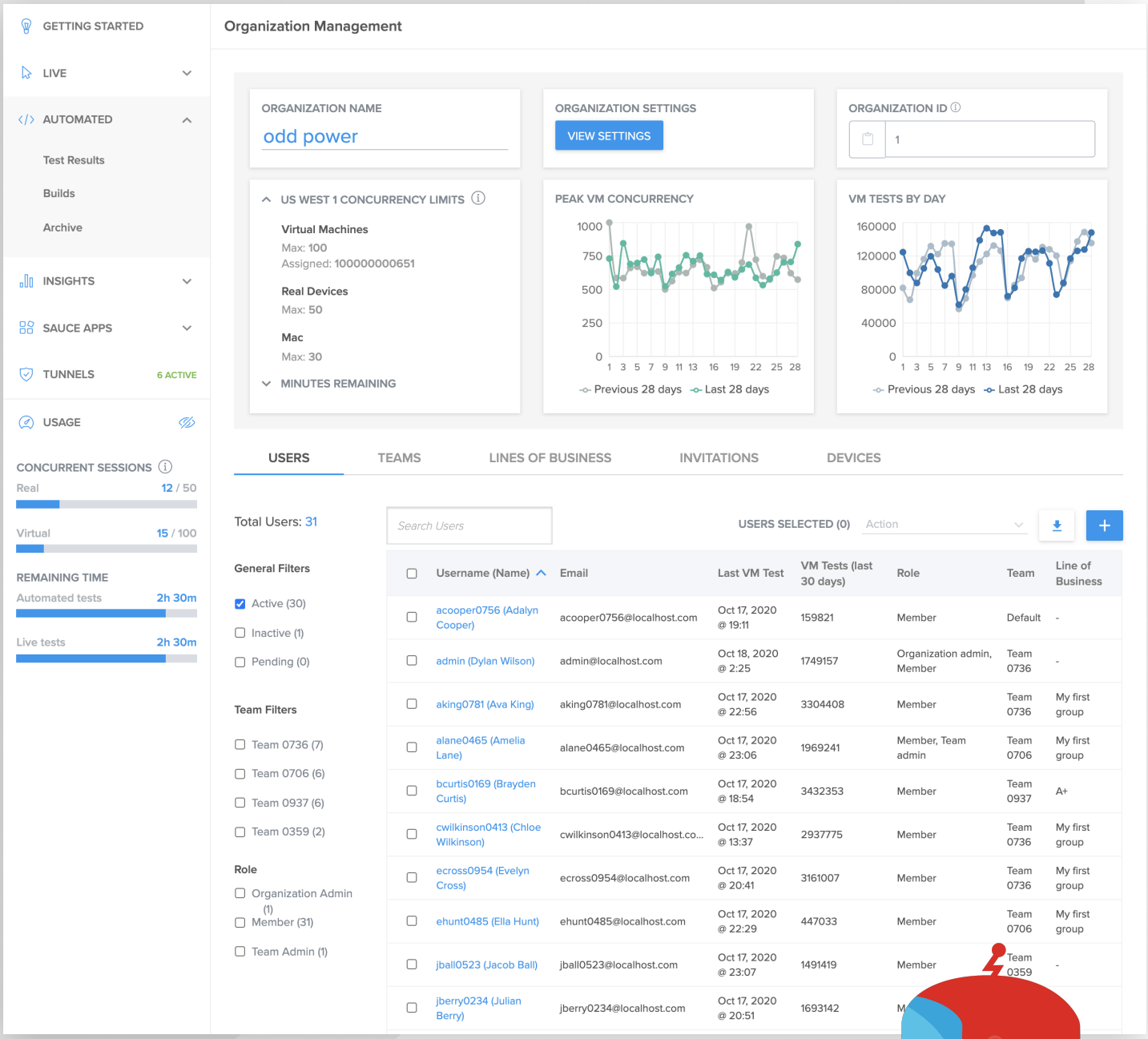
Performance



A more detailed report is also generated and provides a detailed look at the test performance.

Team / User Management

Extended Team Management (xTM) from Sauce Labs provides you with an easy way to manage users and teams, allocate concurrency, and get usage information.

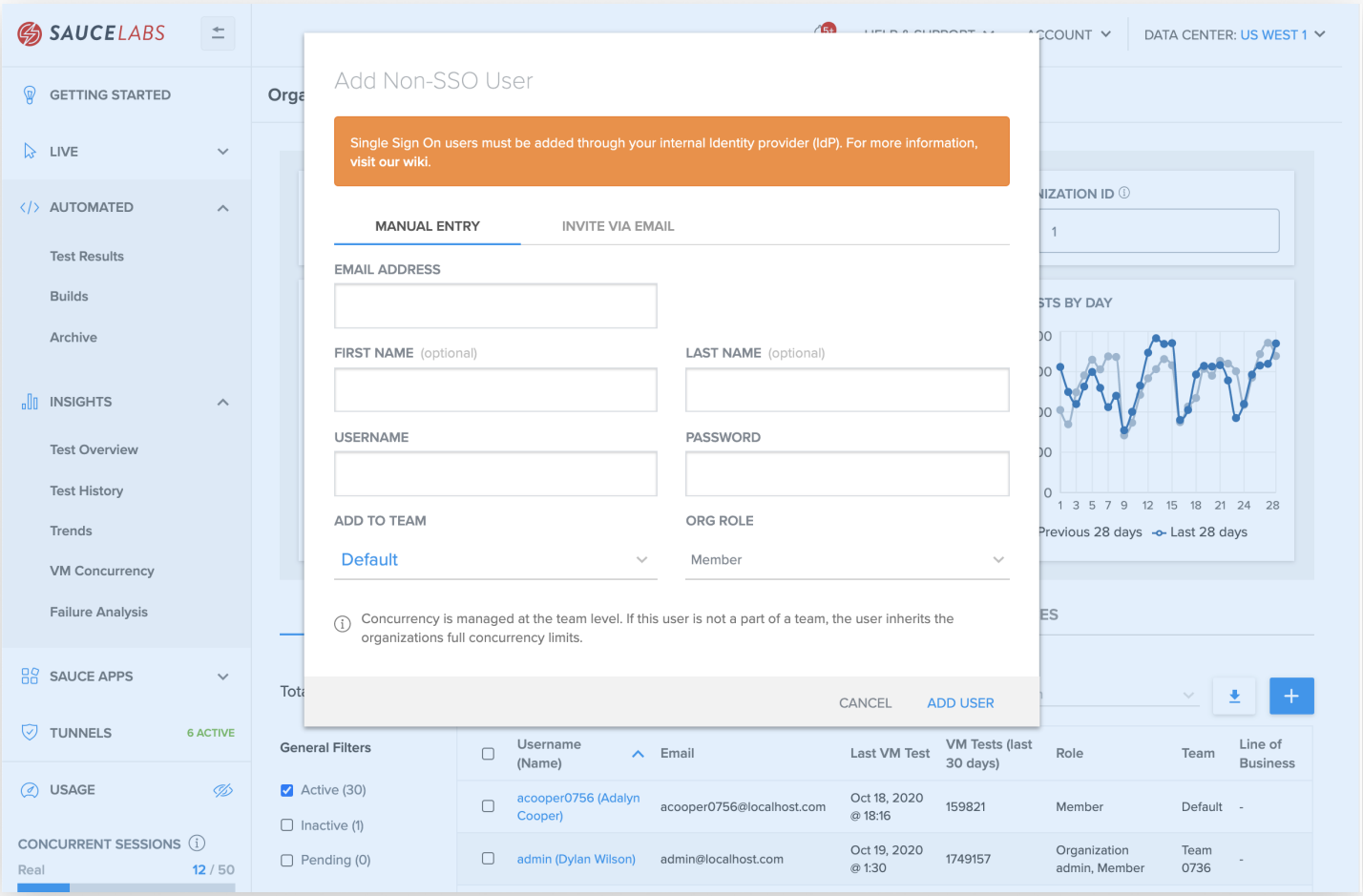


Team Managment - User Management

If you are an organization administrator, you can add users to your organization or team by inviting them via email, or by adding them directly to the organization or team.

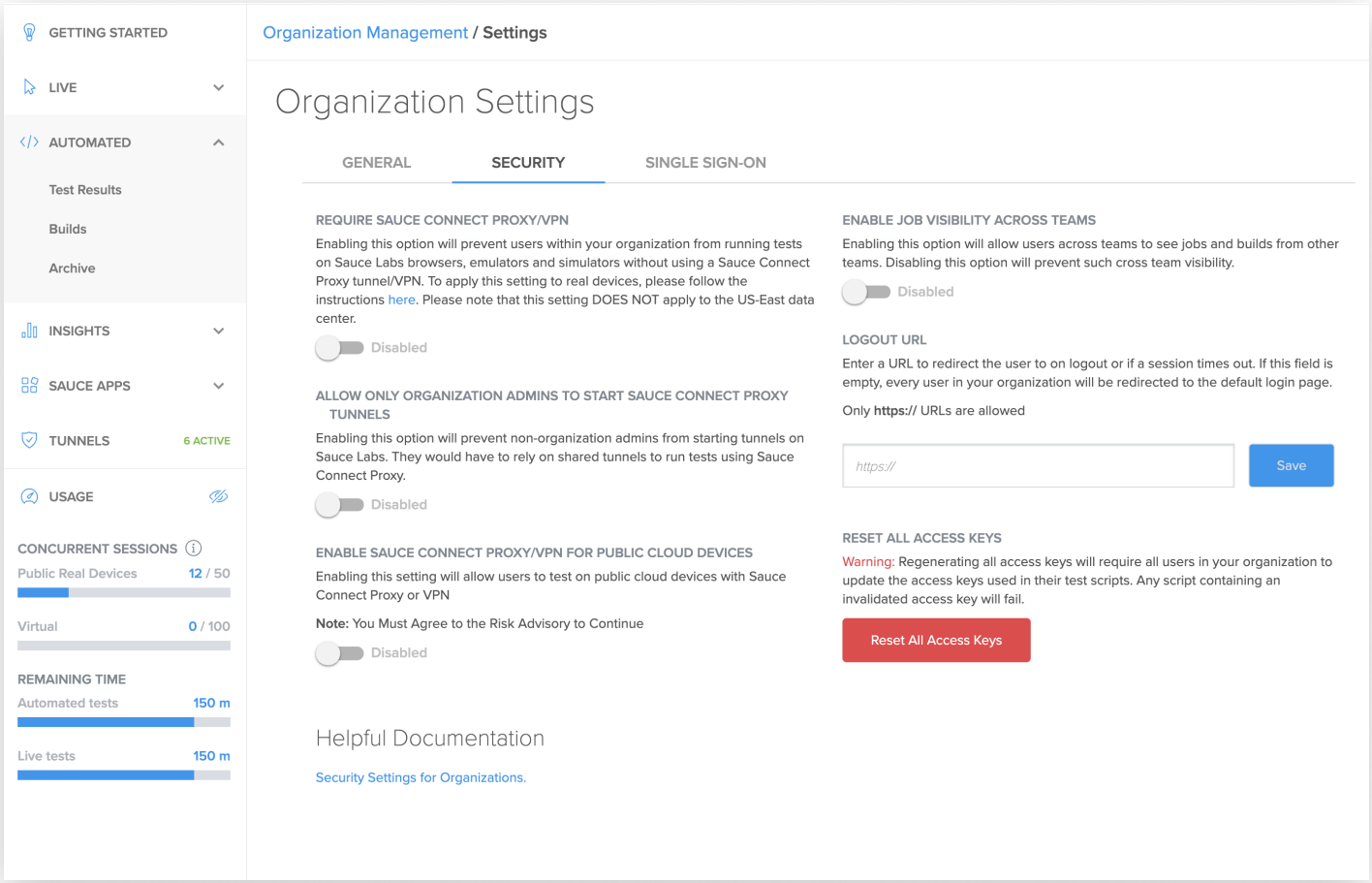
Please note: This feature is NOT enabled when SSO and "Require SSO" are enabled. In this scenario, SauceLabs anticipates that all human users are managed via the customers' Idp, and therefore there should not be a way to add them to the team outside of the customers Idp.

Adding a new user



Organizational Settings

Sauce Labs offers many options for managing user accounts, security, access to job information, access to third-party integrations, and single sign-on.
Read More



Company

Sauce Labs is the leading provider of continuous testing solutions that deliver digital confidence. The Sauce Labs Continuous Testing Cloud delivers a 360-degree view of a customer's application experience, ensuring that web and mobile applications look, function, and perform exactly as they should on every browser, OS, and device, every single time.

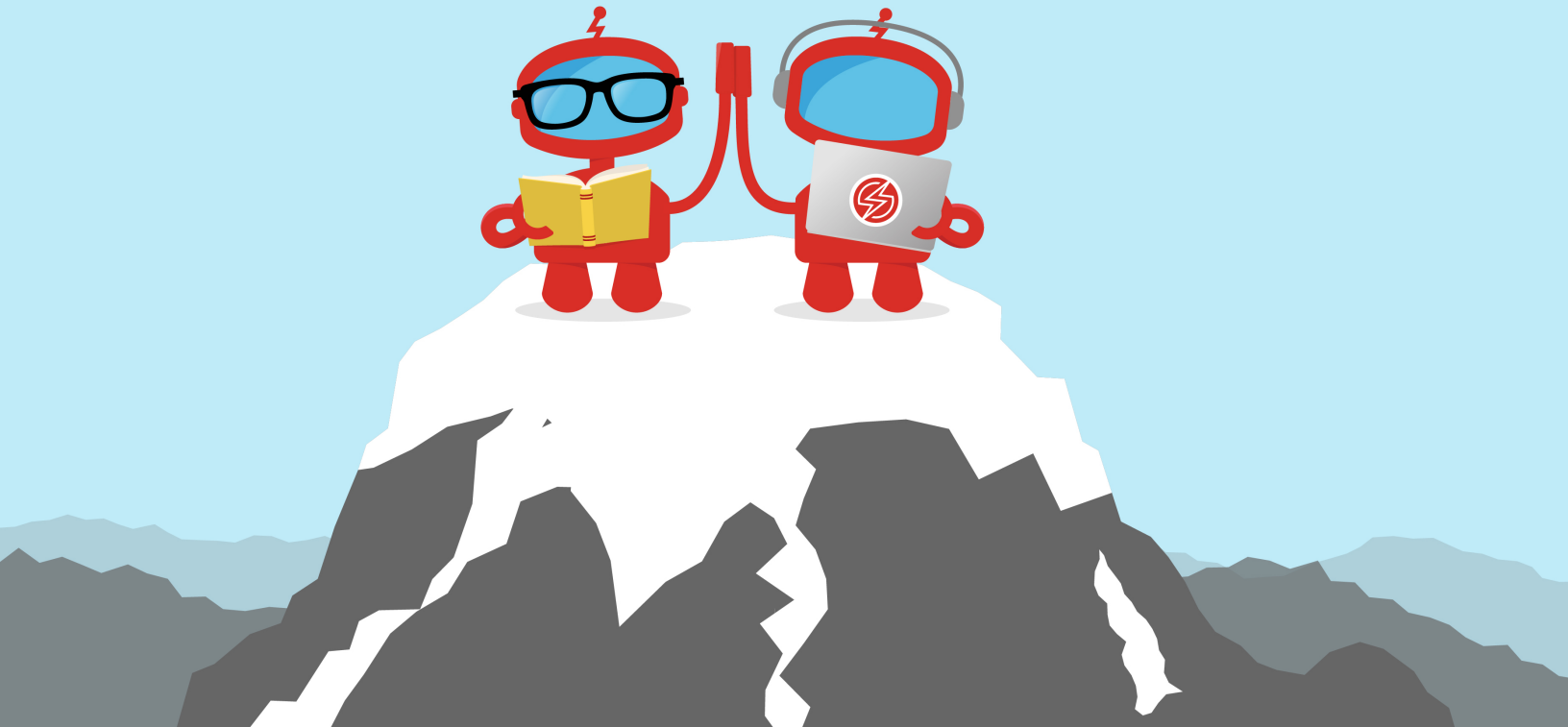
Locations

Sauce Labs - USA (HQ)
116 New Montgomery St, 3rd Floor
San Francisco, CA
94105 USA

Sauce Labs - Berlin
c/o WeWork
Stralauer Allee 6
10245 Berlin

Sauce Labs - Canada
128 W Pender St, 8th floor
Vancouver, BC V6B 1R8

Warsaw
Sauce Labs - Warsaw
Lumen office, Złota 59 St., 4th floor
00-120, Warsaw, Poland



Company Values

Our Values are the "Soul of Sauce". It's how we remain awesome while achieving our mission. It is the unifying force that defines who we are, what we strive to be, and how we delight our customers.

Transparency

We collaborate in a trusting, safe environment. We share knowledge openly as a team to increase our collective understanding and adapt our plans when needed.

Respect Individuality

We celebrate inclusion of different perspectives and experiences. Together, we grow and learn as we treat each other professionally while preserving our individuality (no strict dress codes here).

Constant Improvement

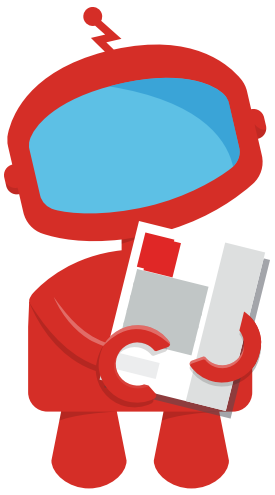
We're committed to continual improvement, even if that requires radical changes. We're not afraid to try new things and support our employees in their professional and personal growth.

Ownership Mentality

We're truly invested in our vision and our impact. We consistently act with our customers' and company's best interests in mind. We take pride in our jobs and work hard but take time to recharge.

Ideas over Hierarchy

When confronted with challenging decisions, we brainstorm a wide range of possible solutions. We test everyone's proposals objectively, without regard to one's title, so the best idea emerges.



Executive Team

ALED MILES
Chief Executive Officer

PAUL JOACHIM
Chief Financial Officer

TERRI AVNAIM
Chief Customer & Marketing Officer

JOE AMADEA
Chief Revenue Officer

JOHN KELLY
Chief Technology Officer

MATT WYMAN
Chief Product Officer

JUSTIN DOLLY
Chief Security Officer

MATT BRUUN
SVP, EMEA

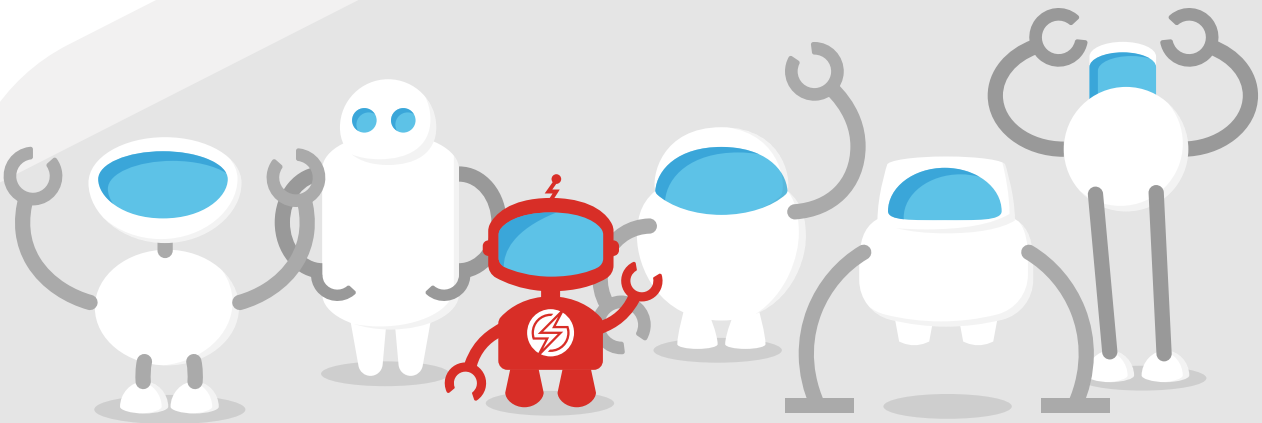
DIANA ECKER-JAMMI
Sr. Director, People

ERIC DEEDS
General Counsel

Full Org Chart

To view the full interactive org. chart, you may log into your Bamboo HR account.

https://saucelabs.bamboohr.com



Slack Channels

#1-sauce-all
Company wide announcements

#it
IT public channel

#pdo-private
Private design team channel

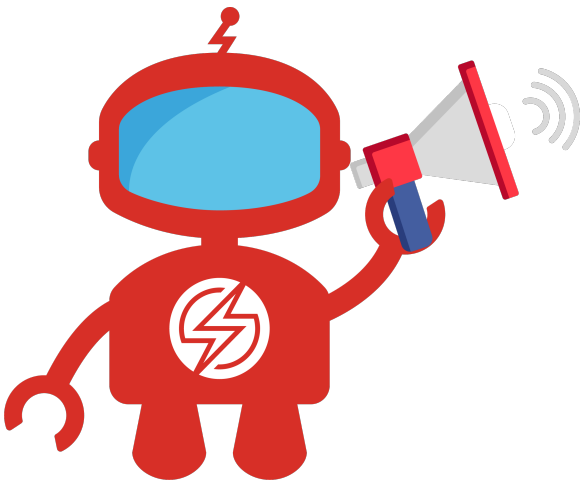
#pdo-public
Product design public channel

#product
General product channel

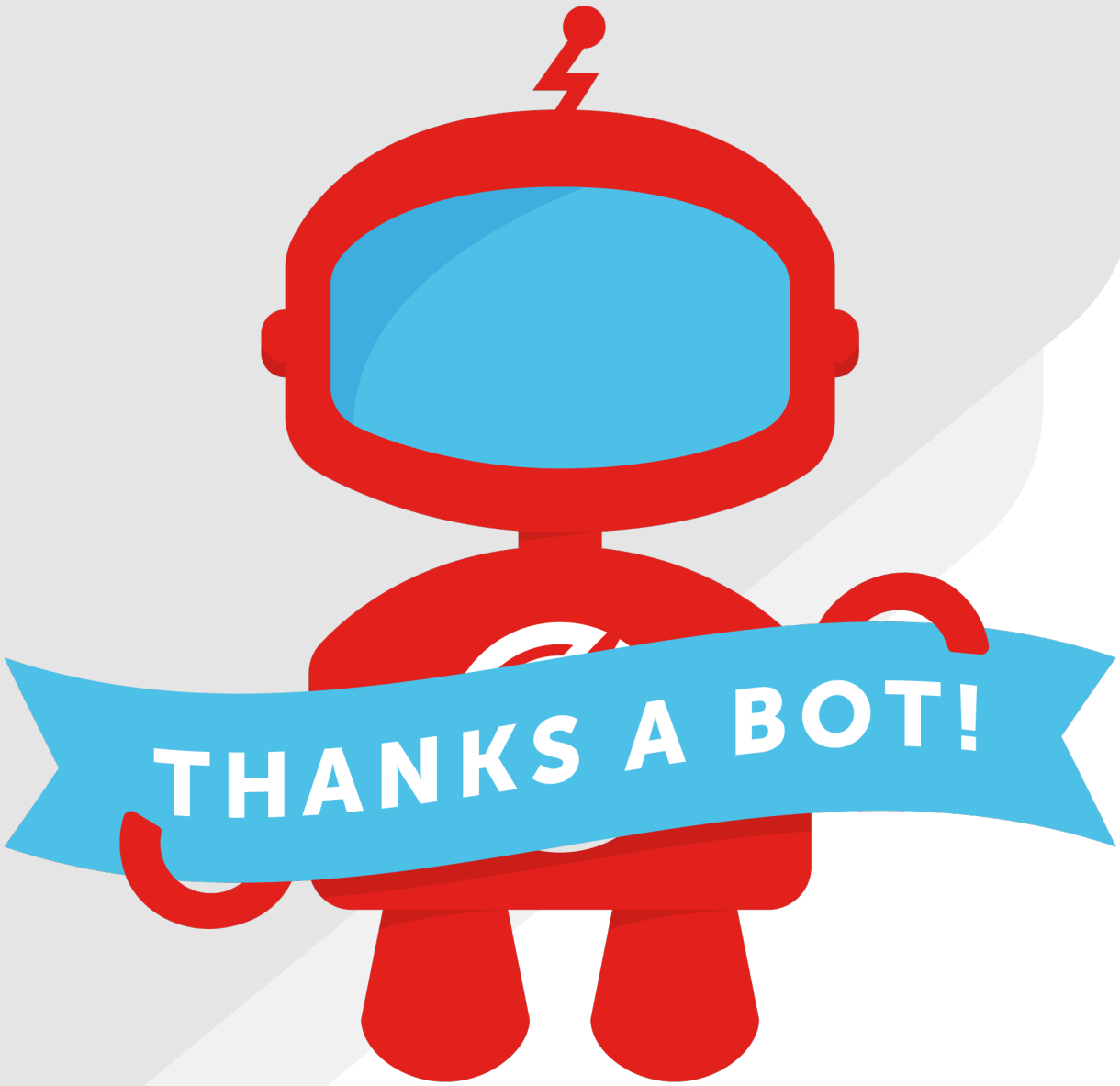
#productteam_private
Product team internal communications

#se-lounge
Interesting customer request and problems

#competition
Information on what our competitors are doing



Fin.



Personas

Developer

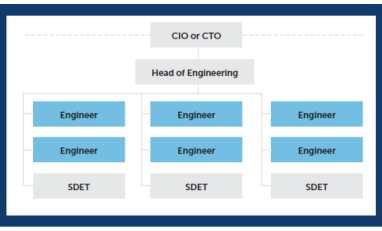
Software Developer Engineer in Test (SDET)

Head of Engineering

Developer

find the original here — Manual QA Persona Card

Developer



Roles & Responsibilities

- I am a Software Engineer or Software Developer.
- My job is to write software and solve customer problems with code.
- It is my responsibility to make sure that my code is free from errors and is well-documented.

Challenges

- It can be difficult for me to know how my commits will affect other teams' code.
- I like staying up to date with technology, but it can be difficult to convince leadership to change.
- Product leadership and my team lead don't always have the same expectations in terms of timetable or development methodology.

Aspirations

Motivation

What keeps me going is knowing that my code **solves real problems** and builds fantastic experiences.

How I'm evaluated

Code coverage goals for testing are often **mandatory,** and my peers will criticize me for writing bad tests. But I don't get praised or promoted for testing more or better.

Stressors

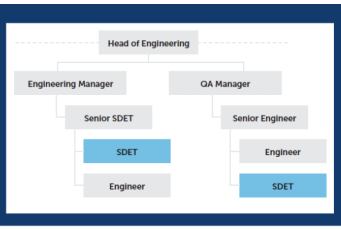
I feel panicked when my **output drops off**, or a bunch of my **commits get rejected** all at once or worse **taking down production**!

SDET

find the original here — Manual QA Persona Card

SDET

Software Development Engineer in Test



Roles & Responsibilities

- I am a Software Quality Engineer, Automation Engineer or QA Engineer
- With a clear understanding of an application's functional and technical goals, I write basic automated tests for repeatable tasks.
- At the end of the software delivery pipeline, I perform tests on new builds and features.

Challenges

- In order to do my job right, I need the build process and tools to integrate seamlessly with testing. But often QA and Dev are organizationally siloed. It's the worst!
- I'm frequently limited by infrastructure constraints and knowledge (about the app) when building out automation.
- I'm tasked with speeding up the SDLC, but manual testing slows down our release cycles.

Aspirations

Motivation

I'm happiest when I can **speed up testing** and **limit flaky tests.**

How I'm evaluated

My bosses are satisfied when I **reduce the time** to test new features and find bugs.

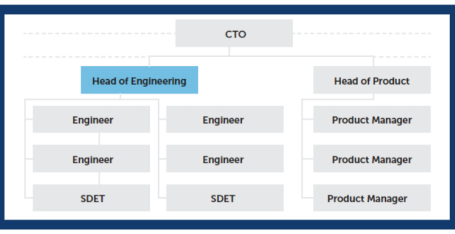
Stressors

Not having enough time to thoroughly test **limits test coverage**. Are we testing the right things?

Head of Engineering

find the original here — Manual QA Persona Card

Head of Engineering



Roles & Responsibilities

- I am a VP of Engineering / Engineering Lead.
- I hire and retain amazing teams with a strong focus on collaboration.
- I deliver great software that drives the growth of the business.

Challenges

- It's my job to report the status of the product back to the executive leadership; how do I pull together the right metrics and analysis to report how we're doing?
- When competitors quickly release new features, we lose market share; how do I create a culture of speed?
- Our business is tied to quality issues reported by customers; how do I increase speed while also reducing risk?

Aspirations

Motivation

I'm happiest when my team and I can **deliver innovative features** to users that **drive growth** for the business.

How I'm evaluated

My bosses are satisfied when I can shorten our product's **time to market** and **deliver value** on a regular **predictable cadence**.

Stressors

With faster cycle times, our old QA methods can't keep up, **testing is our biggest bottleneck**!

Manual QA Tester

find the original here — Manual QA Persona Card

Persona: Manual QA Tester

Roles and Responsibilities

- I am a QA Analyst
- I report to a QA Lead or Engineering Manager
- My job is to ensure software quality through the writing, running and maintenance of tests, and to report issues to relevant teams
- I help influence the selection of testing frameworks and tools for my organization
- Have an eye for UI and UX
- I utilize several software testing concepts: Exploratory and boundary/edge case testing; functional testing; regression testing; unit/integration/UI testing, cyclomatic complexity, test case generation, test report generation, risk analysis & mitigation, team dynamics, etc.
- I'm the link between Product, Development, and the End User. I translate functionality to how an End User would experience our product from many perspectives: Functional; logical; performance; accessibility.

Challenges

- **Perception**: "I'm under intense pressure to ensure quality and not slow down release velocity, which is a challenge when QA is separate and near the end of a sprint cycle. Without an experienced Scrum Master and Dev Team, they may view me as a blocker."
- **Project Ambiguity**: "There is an overall lack of empathy for QA deliverables. Project briefs are often unclear, without definition of what is required. In a perfect world, Devs would write their own tests and be fully responsible for their code since they are the ones who created it. "
- **Scheduling**: "Expectations on how long it'll take for me to test are unrealistic. This happens when QA is not involved from the start, and handled as a separate entity. Requirements are not shared properly with me, or are outdated. And then I'm pressured to finish the evaluation and give the greenlight to go to prod."
- **Limited Tooling**: "When company policies prohibit the downloading and use of certain OS, browser and device versions, then I cannot fully test the application."